LEARN · VERIFY — OFFLINE STUDY BOOKLET

Raising the People Around You

There are not enough hours to help everyone one at a time. Here are the four things you can give a colleague — advice, teaching, guardrails, opportunity — in ascending order of what they cost you and what they build in them, and the three ranges each one can be fired at: one person, a group, or a change that outlives your involvement.

1

Telling people things

Advice: whether it was asked for, why it may not transfer, and how to make it reach further than a conversation.

2

Making it stick

Teaching: the difference from advice, the working-together spectrum, review as instruction, and coaching.

3

Making it safe to move fast

Guardrails: review as protection, project guardrails, processes and robots, and culture change.

4

Handing over the work

Opportunity: delegation, sponsorship, sharing the spotlight, and planning to give away your job.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

> **THE RULE THAT DOES MOST OF THE WORK**
>
> If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

> **WHAT THIS SUBJECT ASKS OF YOU THAT OTHERS DON'T**
>
> Everything here is a **choice between four options that are not interchangeable**, and the commonest mistake is not doing the wrong thing but doing the cheap thing when the situation called for the expensive one. Telling someone the answer, teaching them to reach it, standing behind them while they try, and handing them the whole problem are four different acts with four different results, and only one of them is quick. So read this the way you would read a set of instruments rather than a set of virtues: for each idea, ask **what it costs you, what it builds in the other person, and how far it travels**. Two habits keep you honest. First, resist the pull of the far-right column — the frameworks and programmes look like the important work and usually are not. Second, notice that half of this material is about *restraint*: not answering, not stepping in, not being the one who speaks. That half is harder than it reads.

The spacing schedule

Do a part, then review it after **1 day**, **3 days**, **7 days**, **16 days** and **35 days**. A review is not rereading: it is covering the page and answering the questions in Section 10's tracker. On a fail, that row goes back to a 1-day interval.

What's in here

SECTION	CONTENTS
1	Learning goal, the core model in one paragraph, and the four-part roadmap.
2–5	One part each, always in the same order: plain version → everyday analogy → term table with boundaries → trade-offs → concept-boundary box → anchor sketch → plain/expert phrasing → two written questions on cases that are not in the teaching → a 2×2 → a carry-out rule.
6	The whole subject as one concept sketch, with a question on every node, plus the master 2×2.
7	Symptom → diagnosis → move, and the reverse mapping: you see a practice, what is it for?
8	Numbers worth remembering, and the glossary.
9	Answer key for the eight written questions.
10	Spaced-review tracker and the final quiz.

SECTION 1

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Faced with a colleague, a team or an organisation that is not working the way you would like, you can:

1. **Choose the mechanism that fits**, saying which of advice, teaching, guardrails or opportunity the situation actually calls for, and what each one costs you and builds in the other person.
2. **Choose the range** — one person, a group, or a change that keeps running once you stop — and say why the widest range is usually the wrong first move.
3. **Give advice and reviews that land**: know whether it was asked for, answer the question that was implied rather than only the one that was spoken, and say why the same words help one person and damage another.
4. **Hand work over properly**, choosing who to delegate to and sponsor, describing the guardrails you will provide, and naming the behaviours that quietly take the work back.

The core model in one paragraph

The whole subject starts from an arithmetic problem. Influence begins in individual relationships — reviewing someone's change, answering their questions, mentoring a new joiner — and as your scope grows **there are simply not enough hours in the day** to keep having enough of those. So influence has to be described along two axes rather than one. The first is **what you give**, and there are four things, in ascending order of what they cost you and what they build in the other person: **advice**, which passes on your experience and can be taken or left; **teaching**, which is not aimed at the other person receiving the information but at them *internalising* it; **guardrails**, which do not tell anyone what to do but make it safe to move quickly and to be wrong; and **opportunity**, which is the one that actually develops people, because people learn by doing more than they learn from being told, shown or coached. The second axis is **how far it travels**: **individual**, where you work in a way that grows one person's skills; **group**, where the same effort reaches many at once; and **catalyst**, where the change continues after you step away. Two warnings come attached and both cut against instinct. **Do not skip the small ones** — levelling one person up through review or sponsorship ripples outward, and even world-changing technologists still mentor individuals. And **do not chase the catalyst column**: too many programmes and frameworks overwhelm an organisation, and joining an existing effort usually beats founding a new one. **Do the individual and group work first, and only go broader when the need and the value are very clear.**

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- You can only help so many people one at a time. Past a point, the day runs out.
- There are four things you can give someone, and they get more expensive as you go: tell them, teach them, catch them if they fall, or hand them the job.
- Handing them the job is the one that actually makes them better, because people learn by doing.
- Each of the four can be aimed at one person, at a group, or set up so it keeps working after you've gone.
- But don't rush to the big version. Too many programmes is its own problem, and joining one that exists usually beats starting another.
- Advice is about you, so it might not fit them. Check whether they even wanted it.
- A lot of this is about holding back: not answering, not stepping in, not being the person who speaks.
- The goal is to work yourself out of the job. If the people you helped keep helping others, it carries on without you.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE
1 Telling people things	Why raising other people is part of the job; the three tiers and the four mechanisms; solicited versus unsolicited advice; mentorship; answering the question that was implied; honest feedback and the two audiences of a peer review; why advice that worked for you may damage someone else; and scaling advice by writing, speaking, and building advice flows that do not need you.	First because it is the cheapest and the most tempting. Almost every failure later in the booklet is advice being used where one of the other three was needed.
2 Making it stick	What separates teaching from advice; setting a goal for a session and unlocking a topic; the spectrum from shadowing through pairing to reverse shadowing; six rules for reviewing in order to teach; coaching and its three skills; scaling with classes and self-paced walkthroughs; and the catalyst move of teaching other people to teach.	Second, because it is the same act as advice aimed at a different target — understanding rather than transmission — and everything in it costs more time for a more durable result.
3 Making it safe to move fast	Guardrails as the thing that permits autonomy; the six categories a real review looks for; being a project guardrail; scaling through processes, written decisions, and putting the right thing in people's faces; and the hardest and most effective guardrail of all, which is culture change.	Third, because advice and teaching both put the work of remembering on the learner. A guardrail moves that work into the environment, which is what lets people move quickly without you.
4 Handing over the work	Why experience beats instruction; delegating a messy project rather than a wrapped gift; the behaviours that silently take the work back; sponsorship and its four parts; connecting people to information; sharing the spotlight; and the warning against disappearing into a support role.	Last, because it is the most powerful and the most costly. It is also the only one that ends with you having less work rather than more.

> *“Other people's growth is your growth.”*

1

PART 1 OF 4

Telling people things

The cheapest form of help, what it is worth, when it is unwelcome, and how to make it reach past one conversation.

TENTH-GRADER VERSION

- Helping other people get better isn't a nice extra. Better colleagues means your own work gets easier, and it's how new ways of doing things ever get adopted.
- Advice is you saying what worked for you. That's its strength and its limit — what worked for you might backfire for someone in a different position.
- Check whether they wanted it. If you're not sure, ask.
- Answer the question behind the question. If someone asks whether one thing works and you know the thing that does, say so — don't make them spend an hour finding out.
- Be honest in reviews and feedback. Only saying nice things wastes the person's time.
- Sometimes people don't want a solution at all. They want to say it out loud, or to hear that it's genuinely hard. Ask which.
- If lots of people need the same thing, write it down or say it once to a room instead of repeating yourself.

THE EVERYDAY VERSION

A new driver joins a family and asks how to get across the city at rush hour. The experienced driver has three options and picks the fastest: they describe the route they always take. That is advice, and it is genuinely useful — it is thirty years of trial and error compressed into two minutes. But look at what is hidden inside it. The route works because the experienced driver leaves at half past six, drives a small car, and knows which lane the left turn needs; give the same route to somebody leaving at eight in a van and it is worse than no route at all. Notice, too, what happens if the new driver actually wanted something else. Perhaps they wanted to be told the traffic really is that bad and they are not driving badly. Perhaps they were talking it through to work it out for themselves and would have got there in another minute. **Answering the question they did not ask ends all of those** — so the useful first move is not the route, it is a question: do you want help with this, or do you want to think out loud at me?

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH

Why this is part of the job	Three reasons, and a fourth that arrives at the end.	**1.** Good engineering inevitably goes beyond yourself: even the most powerful virtuoso meets problems too big to solve alone, so better colleagues mean easier work, better software and better business outcomes. **2.** The industry keeps changing, and getting teams to adopt the next game-changing architecture, tool or process is an exercise in frustration unless you know how to influence and teach. **3.** It is the right thing to do — teach your midlevel colleagues well and consider the midlevel engineers *they* will be teaching in ten years.	vs generosity. The third reason has a specific shape worth keeping: **you are sending high standards into the future**. And the fourth, stated at the end rather than the beginning, is the selfish one: **other people's growth is your growth** — if you can delegate you can take responsibility for bigger and more difficult problems, so the more your colleagues can do, the more you can do.
The three tiers	How far a given act of influence reaches.	**Individual** — you are working in a way that grows another person's skills. **Group** — you are scaling your influence by bringing new skills or a change of approach to several people at once. **Catalyst** — the change goes beyond your direct influence, because you have set up frameworks or community structures that let your positive influence continue **even after you step away**.	vs a ladder you climb and leave behind. Two explicit warnings. **Do not skip the smaller ones**: levelling up colleagues through review or sponsorship has a ripple effect, and even the most world-changing technologists still mentor people they believe are worth investing in. And **do not chase the catalyst tier** — see the next row.
The four mechanisms	What you can actually give someone. Advice, teaching, guardrails and opportunity — **a list of options available to you, not a checklist to complete**.	Each exists at all three tiers. Advice runs from mentoring and feedback, through talks and documentation, to mentorship programmes. Teaching runs from review, coaching and pairing, through classes, to teaching people to teach. Guardrails run from review, through processes and automation, to culture change. Opportunity runs from delegation and sponsorship, through sharing the spotlight, to a culture of opportunity.	The instruction is to **play to your strengths and do the ones you enjoy, find easy, or want to get better at**. And there is a hard warning about the widest tier: too many programmes and frameworks can be overwhelming for an organisation, and **you will often have more impact by taking part in an existing initiative than by setting up something new** — teaching a class in an existing curriculum beats founding a separate education effort, and navigating one team through a difficult design can matter far more than tinkering with the review process. **Do the individual and group work first, and only go broader if the need and value are very clear.**

Solicited and unsolicited advice	**Solicited** is when someone asks for something — a recommendation, feedback on their work, help deciding. **Unsolicited** is when they have not asked.	Before offering your thoughts, consider whether the other person is asking for them, and **whether you even have enough context to tell them something both helpful and non-obvious**. If you are unsure whether it will be welcome, **ask**. Two cases where the unsolicited version earns its place: when the person is in a situation they cannot see out of, and when **you have key information they do not have** — that the platform they are about to adopt is being turned off next year.	vs candour. The opening maxim is that free advice is worth exactly what you paid for it, and the reason is structural: **advice is noisy — there is bad signal mixed in with the good, and it tends not to be tailored to the person receiving it**. Consider your role and your relationship too, since some advice should come from friends rather than strangers. Do not launch in: ask whether you may offer some, and get permission. And if you are itching to give advice on a topic nobody is asking you about, **write it publicly instead**.
Mentorship	Sharing your experience so someone can make use of it themselves — often an engineer's first experience of leadership.	It can be formal, through an assigned pairing, or entirely organic: sit next to someone, introduce yourself, answer their questions, and you may find you have an informal mentee. Set it up for success — agree that you will meet, say, weekly for six weeks, and agree what you are trying to achieve, whether that is settling into a new company, learning a codebase, or working toward a new role.	vs coaching, which is in Part 2 and teaches people to solve problems for themselves. **Mentoring is focused on your experience**, which is exactly its limit. It is also **not just for new people** — mentees may have decades of experience — and **not necessarily one-way**: your mentee may give you a new perspective or teach you things they know better than you.
Advice that does not transfer	The failure mode built into mentoring: **the advice that worked for you might not work for someone else**.	Guidance like “be louder in meetings” or “ask your boss for a raise” **may undermine someone else, because members of underrepresented groups are unconsciously assessed and treated differently**. Best practices from a decade ago may not work for a younger colleague now, and the social dynamics or the technology of your experience may not map onto theirs. “Just message the director and ask to be invited” is easy when the director is your peer and much harder when they are three levels above you.	vs bad advice. This is *correct* advice, aimed wrong. The same problem recurs in written feedback, most often about communication style: **“be aggressive” will make some people seem more like leaders and get others into trouble**. So watch for implicit bias in how you describe the same behaviour in different people — was it consensus building or indecisiveness, refreshingly down to earth or unprofessional? **Often it depends on who you are talking about.**

Answering the implied question	Working out **what advice is being implicitly solicited**, even when it is not directly asked for.	The junior engineer asks whether one endpoint returns a full name; told only “no”, they spend an hour working around it before nervously asking whether some other endpoint would — and of course one does. **The senior person had the information and it did not occur to them to impart it.** If you are unsure, ask what your colleague is trying to do and whether they need help.	vs infodumping — offloading every snippet that could connect to the topic, which is explicitly not the instruction. There is a matching precision rule for review comments: be clear whether you are **sharing interesting information or asking them to change course**, because a bare “this other library would also work here” leaves the author guessing whether you hate the current approach. And the knowledge worth passing on is not only technical: **how to navigate the organisation and get things done is valuable knowledge too**.
Do you want to vent, or do you want advice?	The question that prevents the commonest mentoring error.	When someone describes a difficult project, the easy and intuitive response is to say what you would do. It is kinder, if harder, to work out what they actually need. They may want help — or **reassurance that other people would also find this difficult**; or they may be talking it through to unpack it for themselves, which is **rubber duck debugging**; or they may want to tell a war story and hear commiseration for what they have navigated so far.	**Unsolicited advice derails all of those things.** The move is to ask which is wanted and then supply either comfort and validation or solutions, as requested — not to guess. Note this is a failure of kindness rather than of accuracy: the advice can be perfectly good and still be the wrong thing to have said.
Honest feedback	Telling the truth about someone's work when they have asked you to look at it — the good *and* the parts that did not land.	Reviewing a colleague's talk means noting what was funny, insightful or educational, and also **anything that did not work, anything you thought was not correct, and anywhere you started to tune out**. Otherwise you are wasting the presenter's time.	vs criticism. Do call out the sections you think are great — and **pay them the respect of being kindly honest**. It is acknowledged to be hard: teasing out exactly what is not working and finding words for why takes effort and makes for a more difficult conversation. **But you will not help your colleague if you hide the truth.**
The two audiences of a peer review	Written performance feedback is read by **the person who asked for it** and by **their manager and others calibrating or evaluating them**.	For the first audience: assume they genuinely want to know how to improve, and take “what could this person do better?” literally rather than as an invitation to criticise. If you cannot think of anything, **ask yourself why they are not one level more senior — or two — and give advice on the behaviours to focus on to get there**. For the second: give them what they need to help your colleague grow and to notice patterns that need addressing.	The tension between the two audiences is the point. Think about how your words will be perceived and **whether they can be taken out of context by someone who does not know the work you are describing**. And there is a concrete escape hatch: if you find yourself holding back on an area for growth because you do not want to accidentally torpedo a well-deserved promotion, **deliver that feedback privately instead**.

Scaling advice: writing and a microphone	Two ways the same effort reaches a group rather than a person.	**Write it down**, so you do not explain the same thing again and again — a set of frequently asked questions, a how-to, even a descriptive channel topic lets you say something once and have it read by many; if it applies outside your company, publish it. **Look for opportunities to get a microphone and an audience**: a slot at an all-hands, a conference, a talk event. One group who wanted to raise testing quality wrote their advice as one-page sheets and posted them in toilet stalls — the ultimate in unsolicited advice, and people read it.	The microphone has a use that is easy to miss: you can sometimes deliver **the message you are focused on alongside the message you were asked to share**. Invited to present on an incident of your choice, you can pick one that carries the point you are trying to make — an outage made much worse by a circular dependency becomes the vehicle for a message about being deliberate in what you depend on. **An audience and a microphone at the right time.**
Advice flows that do not need you	The catalyst tier for advice: **make it easy for your colleagues to help each other**.	If your organisation relies on one-to-one conversations to understand how anything works, encouraging people to write things down is among the most powerful things you can do — but **start small**, because going from an oral culture to writing everything down will not win you any friends. Look for a small but meaningful change: an easy-to-use documentation platform, a page of answers to the questions a team is interrupted by most, a quarterly documentation day with a director's endorsement.	Every catalyst move here carries a cost that is bigger than it looks. A monthly talk series is **a bigger commitment than just scheduling the meeting** — you must solicit talks, send reminders and possibly watch practice runs, so plan for it and **ideally start with at least three people to share the load**. A mentorship programme's administrative work will be more time-consuming than you expect and **is generally not considered part of an engineer's job**; a manager may find it easier to frame as part of theirs — and **convincing someone else to set it up still counts as being a catalyst**.

CONCEPT BOUNDARY — ADVICE IS ABOUT YOU, AND THAT IS BOTH ITS VALUE AND ITS CEILING

What it sounds like: the generic word for helping someone — the thing you are doing whenever you say something useful to a colleague.

What it actually is: one specific mechanism with a defining property. When you give advice you are explaining *how you relate to the topic*, and the receiver can take it or leave it. That is why it is cheap, why it scales through writing and speaking, and why it is the only one of the four you can perform without knowing much about the person in front of you.

Why that matters practically: every limit of advice follows from that one property. It may not transfer, because your position was not their position. It may be unwelcome, because you were not asked to relate to their topic at all. It is noisy, because it is untailored by construction. And it stops at the point where the other person needs to be able to do the thing rather than know about it — which is precisely where the next three mechanisms start.

The trap: reaching for it because it is fast. The alternative to bad advice is usually not better advice; it is a question. Do you want to vent or do you want advice? What are you actually trying to do? Would it help if I sat with you while you tried it? Each of those buys information you did not have, and the last one has already stopped being advice.

Anchor sketch — the two axes, and what advice can and cannot do

Legend: bold = a named tier, mechanism or rule **reversed = the fact everything else answers** **Q = cover the page and answer this**

THERE ARE NOT ENOUGH HOURS IN THE DAY TO HELP EVERYONE ONE AT A TIME.

so influence needs TWO axes: what you give,
and how far it reaches.

|

WHY BOTHER . three reasons, and a fourth at the end

1 good engineering GOES BEYOND YOURSELF. even the virtuoso meets problems too big to solve alone.

2 the industry keeps CHANGING. adoption is misery without the skill to influence and teach.

3 it is right. teach your midlevels well and think of the midlevels THEY teach in ten years: YOU ARE SENDING HIGH STANDARDS INTO THE FUTURE.

4 (stated last) OTHER PEOPLE'S GROWTH IS YOUR GROWTH. delegate, and you can take on bigger problems.

AXIS 1 - FOUR MECHANISMS . rising cost, rising effect

ADVICE you explain how YOU relate to the topic. take it or leave it.

TEACHING aimed at the other person INTERNALISING it.

GUARDRAILS not instruction. permission to move fast.

OPPORTUNITY people learn by DOING, more than from teaching or coaching or advice.

-> a list of OPTIONS, not a checklist. play to your strengths.

AXIS 2 - THREE TIERS . how far it travels

INDIVIDUAL you work so that one person grows.

GROUP one effort reaches many at once.

CATALYST it CONTINUES AFTER YOU STEP AWAY.

-> do NOT skip the small ones. review and sponsorship ripple outward.

-> do NOT chase the catalyst column. too many programmes overwhelm an org, and JOINING an existing effort usually beats founding one. individual and group FIRST.

ADVICE, UP CLOSE

WAS IT ASKED FOR? solicited = they asked. otherwise check: do you have the context to say something helpful AND non-obvious? if unsure, ASK.

two cases that earn the unsolicited version:

they cannot see out of the situation . you hold KEY INFORMATION they lack.

-> itching to say it and nobody asked? publish it.

MENTORING IS FOCUSED ON YOUR EXPERIENCE -- its value and its ceiling. it is not only for new people and not only one-way. set expectations up front.

ADVICE CAN BE RIGHT AND STILL AIMED WRONG . "be louder" may UNDERMINE someone assessed differently.

old best practice may not fit a younger colleague.

"just DM the director" depends who the director is.

same trap in reviews: consensus building or indecisive? down to earth or unprofessional?

```
ANSWER THE IMPLIED QUESTION . "no, that endpoint
     gives the username" cost an hour. the information
     was there and was not offered.
     -> but do NOT infodump. and say whether a comment
        is INTERESTING or a REQUEST TO CHANGE COURSE.
VENT OR ADVICE? they may want reassurance that it IS
     hard, or a rubber duck, or a war story.
     unsolicited advice derails all three. ask first.
BE HONEST . name what did not work, what seemed wrong,
     where you tuned out. hiding it wastes their time.
PEER REVIEW HAS TWO AUDIENCES . the person (take
     "what could they do better" LITERALLY -- why are
     they not a level up?) and the calibrators (can
     your words be taken out of context?).
     -> holding back to protect a promotion? say it
        PRIVATELY instead.
-----------------------------------------------------
SCALING IT
GROUP . write it down (say it once, read by many) .
     get a microphone -- and carry YOUR message
     alongside the one you were asked to deliver.
CATALYST . advice flows that do not need you. START
     SMALL: an oral culture will not write everything
     down. a talk series needs >=3 people to share the
     load. a mentorship programme is admin nobody counts
     as engineering -- and convincing SOMEONE ELSE to
     run it still counts as being a catalyst.
```

Q Name the four mechanisms and the three tiers, and give the two warnings attached to the tiers.

Q What is the defining property of advice, and which of its limits follow from it?

Q Give the three things a person might want instead of advice, and the question that tells you which.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Giving advice rather than teaching or coaching	Speed, and the compression of your experience into a few sentences.	It is untailored by construction, may not transfer, and leaves the person no better able to solve the next one.	When the gap is information they lack rather than a skill they need, and when they asked.
Offering advice nobody asked for	Occasionally the thing they most needed to hear — the unhealthy situation, the platform being turned off.	It may be unwelcome, and you may not have the context to say anything both helpful and non-obvious.	When you hold key information they do not, or they cannot see out of the situation — and ask permission first.
Asking “do you want to vent or do you want advice?”	You find out which of four different things is actually wanted before you spend the conversation on the wrong one.	A moment of awkwardness, and giving up the satisfaction of having the answer.	Every time somebody starts describing a difficult situation. The intuitive response is the wrong one here.

Being kindly honest in a review of someone's work	They can actually improve, and your praise starts to mean something.	Real effort to tease out what is not working, and a harder conversation.	Whenever they asked you to look. Withholding the truth is not a kindness; it wastes their time.
Writing an area for growth into a peer review	The manager and the calibrators get what they need to help your colleague grow and to spot patterns.	Words that can be taken out of context by people who do not know the work.	When it will be read as it is meant. If it might torpedo a deserved promotion, deliver it privately instead.
Writing it down instead of repeating it	You explain it once and it is read many times, by people you would never have reached.	Writing time now, and maintenance later.	The second time you find yourself explaining the same thing.
Setting up a talk series or a mentorship programme	An advice flow that keeps running without your involvement in each instance.	Soliciting, reminding, reviewing, administering — and for a mentorship programme, work not generally counted as an engineer's job.	Only when the individual and group work is already done and the need is clear. Consider joining or convincing someone else to run it.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“Let me tell you what I'd do.”	“Do you want to vent, or are you looking for advice? Either is fine — I just don't want to answer the wrong question.”
“I told them what worked for me and they ignored it.”	“My advice was about my position, not theirs. Would the same move have read the same way from where they're standing?”
“They only asked whether that endpoint worked.”	“What were they actually trying to do? I had the answer and didn't offer it — that's an hour of theirs I cost.”
“I left a comment saying another library would work.”	“Was that a note of interest or a request to change course? They can't tell, so I should say which.”
“It was good, I didn't have much to add.”	“Where did I tune out, and what did I think was wrong? Hiding that wastes their time.”
“I couldn't think of anything they should improve.”	“Why aren't they a level more senior — or two? That's the growth advice.”
“She's abrasive; he's refreshingly direct.”	“Am I describing the same behaviour with different words depending on who did it?”
“I've explained this three times this week.”	“Then it's a document, not a conversation. Say it once and let it be read many times.”
“We should start a mentorship programme.”	“That's mostly administration nobody counts as engineering. Is there an existing one to join, or a manager who'd own it?”

PART 1 · QUESTION 1

A staff engineer has a standing weekly slot with a mentee. This week the mentee begins describing a migration that is going badly — three teams disagreeing, a deadline nobody believes — and the engineer, twenty seconds in, explains the approach they used on a similar migration four years ago and suggests messaging the director to get onto the steering call. The mentee goes quiet and the conversation ends early. Separately that week the engineer answers a question in a channel — “does the batch API accept a date range?” — with “no”, and later sees the asker spent the afternoon building a loop, when a different call would have done it in one line. And in a design review they comment only “the streaming client would also work here”, then is surprised when the author rewrites the whole document.

(a) Name the three things the mentee might have wanted instead of advice, and give the question that would have found out. **(b)** The director suggestion may be correct and still wrong. Explain the mechanism, and give a second example of the same failure. **(c)** Name what went wrong with the batch API answer, state the principle it violates, and give the one thing that principle explicitly does not license. **(d)** Explain why the design-review comment produced a rewrite, and give the precision rule that prevents it.

PART 1 · QUESTION 2

An engineer in a 400-person organisation notices that the same six questions about release procedure reach them every week, always by direct message. Their plan, presented to their director, is to launch a company-wide mentorship programme, a monthly talk series and a written-culture initiative requiring every team to document its systems — all starting next month, all run by them. Asked for peer feedback on a colleague in the meantime, they write four sentences of praise and nothing else, privately reasoning that the colleague is up for promotion and any criticism could be misread by the committee. They also describe a second colleague as “indecisive” for behaviour they praised in someone else as “consensus building”.

(a) Name the tier the plan is aimed at, give the two warnings that apply to it, and say what the six questions actually call for. **(b)** Take the three initiatives one at a time: give the specific cost the material attaches to each, and the cheaper alternative where one is offered. **(c)** Evaluate the peer feedback against both of its audiences, and give the escape hatch that resolves the engineer's dilemma. **(d)** Name what the “indecisive” description is an instance of, and give one further example of the same pattern.

2×2 — did they ask, and do you actually know enough

COLUMNS: DO YOU HAVE THE CONTEXT TO SAY SOMETHING HELPFUL *AND* NON-OBVIOUS? →

	Yes — you know their situation and hold something they don't	**No — you know the topic, not their position in it**
They ASKED for it	**Say it, and say it honestly** The one quadrant where advice is simply the right tool. Name what worked and what did not, including the parts that are uncomfortable. *Move:* answer the question they implied as well as the one they asked — and if the answer is a review comment, mark whether it is information or a request to change course.	**Ask before you answer** They asked, and you are about to generalise from a position that was not theirs. This is where correct advice does damage. *Move:* find out what they are actually trying to do, and check whether the move you are about to recommend reads the same way from where they stand.
They did NOT ask	**Offer it, with permission** The two cases that earn unsolicited advice: they cannot see out of the situation, or you hold key information they lack. *Move:* ask whether you may offer some first. Consider your role and relationship — some things should come from a friend rather than a stranger.	**Write it somewhere else** Untailored advice, unrequested, from outside their situation. This is the quadrant the maxim about free advice is describing. *Move:* if you are itching to say it and nobody is asking, publish it where the people who want it can find it — which is also how advice scales.

Read it as: the row is *permission* and the column is *standing*, and the reason both are needed is that each fails differently. Advice without permission is an intrusion; advice without standing is noise that happens to be well meant. The top-left is the only cell where saying the thing is the whole job — everywhere else the first move is a question, and in the bottom-right it is not a conversation at all.

CARRY-OUT RULE FROM THIS PART

Before you answer, find out what was actually being asked — and before you scale, find out whether anyone wanted it. Advice is you explaining how you relate to a topic, which makes it fast, untailored, and easy to aim at the wrong target. The two questions that do most of the work cost one sentence each: *do you want to vent or do you want advice?* and *what are you trying to do?*

2

PART 2 OF 4

Making it stick

What separates teaching from advice, the spectrum of working together, reviewing in order to teach, and coaching.

TENTH-GRADER VERSION

- Advice is telling. Teaching is aiming at the other person actually getting it — so they can do it later without you.
- Decide before you start what they should be able to do by the end. “Draw this system on a whiteboard.” “Send your first change.”
- They have to do things, not just watch. It's better still if they leave with something they made.
- There's a slider: you do it and they watch, you both do it, or they do it and you watch. All three are useful, at different moments.
- Reviews teach. Say *why*, not just *what*, or they won't know what to do next time.
- Mark which of your comments are serious and which are just preferences. Otherwise they can't tell.
- Coaching is the opposite of advice: you ask questions and stay quiet so they find their own answer. It feels wrong and it works better.
- If you write a class, teach someone else to teach it. Then it keeps running without you.

THE EVERYDAY VERSION

Somebody wants to learn to cook one dish. There are four ways this goes. You can recite the recipe at them, which takes four minutes and works about as well as you would expect. You can cook it while they watch and take notes, which shows them what good looks like — the heat you use, the moment you stop stirring — but leaves their hands untrained. You can cook it together, which is slower and lets you notice, in the moment, that they think “fold in” means “stir hard”. Or you can sit at the table with a cup of tea while they cook it and you say nothing unless something is about to burn. Only the last one produces somebody who can cook the dish, and it is also the one where you are most tempted to take the spoon. Notice, too, the difference between telling them to lower the heat and telling them **why** the heat matters: the first fixes tonight's dinner, the second is the only version that survives to the next dish.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH

Teaching	The difference from advice is **understanding**. Giving advice means explaining how you relate to the topic, and the receiver can take it or leave it; teaching means trying to have the other person **not just receive the information but internalise it for themselves**.	It is a step up from advice in what it costs and in what it leaves behind. Everything else in this part is a technique for aiming at internalisation rather than transmission.	vs seniority. **Deliberate teaching is not just for a more senior person helping a more junior one**: it is useful any time someone new joins a team, or whenever you have more domain knowledge than the person you are working with — which cuts both ways. Asking a colleague for an overview of their systems is teaching in your direction, and an hour of whiteboarding leaves you with a much clearer picture of how their systems interact, plus a diagram of your own to keep or to add to their documentation.
Unlocking a topic	What the best classes do: you walk away feeling you have **a handle on something you did not have before** — a new skill or understanding you can build on — and it sparks curiosity and interest.	Teachers have a specific goal, often formalised in a lesson plan, and so should you. Giving an overview of a system? By the end, the person should be able to **draw it on a whiteboard and describe it to someone else**. Walking through a codebase? Aim to give them everything they need to **send their first pull request**. Showing them a tool or interface? **Describe three to five common scenarios they should be able to handle on their own by the end.**	vs covering the material. Each of those goals is stated as a *capability the learner has afterwards*, not as a list of things you said — which is what makes it checkable at the end of the session. Note the shape: the goal is always something they can do without you in the room.
Activating knowledge	Successful teaching includes hands-on learning: **the student should be doing as well as listening**.	Find opportunities to let them lead — using the tool themselves, typing the commands, opening the tabs on their laptop rather than yours. **It is even better if they end up with an "artifact" to refer back to**, such as a diagram or a code snippet.	vs demonstration, which is a different point on the spectrum below and has its own uses. The tell that you have skipped this is a session where the only hands on the keyboard were yours. The artifact matters because it survives the session; the memory of watching does not.
The spectrum of working together	Three points on one line, from you doing everything to them doing everything.	**Shadowing** — you execute the skill and your shadow watches and takes notes on how you are approaching it. **Pairing** — you are still working together but the shadow has become an active participant: pair programming, co-authoring a document, whiteboarding an architecture, solving a problem together. **Reverse shadowing** — the learner performs the task while the experienced person watches and gives notes.	The three are not ranked; **different points on the line will be useful in different situations**. Each buys something distinct: shadowing is **teaching by demonstrating** and a chance to be a visible role model working to high standards; pairing lets you **share knowledge and check for understanding in real time**; and reverse shadowing is where the learning is densest, because **no matter how closely they have paid attention, they learn most by activating the knowledge and practising the task**. Reverse shadowing is also a **guardrail**, which is Part 3.

Reviewing in order to teach	Using code and design review to highlight perils your colleague might not know about, suggest safer alternatives, and **encourage behaviours you want to see more of**.	A review from a senior person **can be a real confidence booster** — and a review done wrong can destroy confidence instead. It is soul-destroying to work through a barrage of comments that are condescending, seem arbitrary, or that you do not understand.	vs reviewing to prevent harm, which is the same activity with a different objective and is treated in Part 3. Here the stated job is to **impart the knowledge and point out problems in a way that retains your student's confidence and growth mindset**. Six rules follow, in the next six rows.
Rule 1 — understand the assignment	Be aware of the context and of the stage of the work.	Is your colleague new to the language or technology and looking to learn, or do they just need a second pair of eyes for safety? And what stage is this? **Reading a first high-level draft, start with the foundations and the approach and stay out of the nitpicky details.** If everyone has bought in and this is the last review before launch, **it is not the time for big directional questions** — get into the weeds and be extra alert for what could go wrong.	vs a fixed personal review style applied to everything. The same comment is helpful or destructive depending on stage: a foundational objection is a gift on a first draft and a catastrophe the day before launch, and a nitpick is the reverse.
Rule 2 — explain why as well as what	A bare instruction tells the author what to do *right now*; it does not tell them what to do next time.	“Don't use this pointer type, use that one” is a fact, not an understanding. The author *could* go and read the documentation on whatever you just told them about, **but they might not recognise why it applies**. A short explanation, or a link to a specific relevant article rather than a general manual, is a shortcut that helps them learn.	vs correctness. The comment is right either way — the difference is entirely in whether it transfers. **Teaching means sharing understanding, not just facts**, and this row is where that distinction becomes an actual editing instruction.
Rule 3 — give an example of what would be better	If a section is confusing, do not simply ask for it to be clearer.	“Please make this more clear” is **hard to know what to do with**. Offer a couple of suggestions of what you think the author is trying to say.	vs writing it for them. Two suggestions of what you think they meant is also a check on your own reading: if neither is right, you have surfaced a misunderstanding that the vague version would have hidden.
Rule 4 — be clear about what matters	When you are less experienced **it can be hard to calibrate the advice you are given**. Some things are vitally important, some are nice to have, and some are just personal preference — so annotate.	The four levels, by example: **“use parameterised queries instead — you are opening yourself up to an injection attack and a malicious user could drop our database”**; **“the way you are approaching this will work fine, but we prefer to avoid that pattern — here is the section of the style guide that says why”**; **“I'd recommend one bigger service rather than two small ones, I think it will be easier to maintain — but I don't feel strongly, your call”**; and **“this is just a nitpick, but the other spellings in this file are American English”**.	vs consistency of tone, which is what makes an unannotated review unreadable: every comment arrives at the same volume, so the reader either treats the nitpick as a blocker or the security hole as an opinion. Note that the second example carries something extra — a link to the written decision that settles it, which is Part 3's territory.

Rule 5 — choose your battles	Review in several passes: **first high-level comments, then in increasing detail**.	The reasoning is blunt: **if the code does not do what it is supposed to, it does not matter whether the indentation is correct**. The same holds for design documents — **if your first comment is that the author is solving the wrong problem, it is not helpful to leave a hundred technical suggestions**.	vs thoroughness, which is what a hundred comments feel like from the reviewer's side. There is a related instruction for the same situation: if the change or document needs a lot of work, **the most constructive approach may not be to bury your colleague in comments** — consider setting up time to talk, or to pair with them, instead.
Rule 6 — if you mean "yes", say "yes"	Make it clear whether your comments are **blocking or not**, and whether you are otherwise happy with the change.	Call out the good as well as the bad, and **explicitly say "this looks good to me" on design documents**. Code review usually ends with a button that says you believe the change is safe to merge; a document has no such button. **When there are a lot of reviewers, each may be hesitant to approve until the others have weighed in** — so if you have no objections, say so.	vs silence, which reads as unresolved rather than as assent. This row also carries the reminder that **engineers earlier in their careers may find you intimidating, or be reluctant to question your suggestions even when they think you are wrong** — so make comments friendly, approachable and human.
Coaching	Where mentoring involves sharing your personal experiences, coaching **teaches people to solve problems for themselves**. It is slower, and your colleague learns much more by making their own connections than by being given the answers.	Three skills, none of them instinctive. **Asking open questions** — ones that cannot be answered yes or no, which yield much more information; dig into the problem rather than starting from a solution, and help the coachee unpack aspects they may not have considered. **Active listening** — reflect back what you heard, both to check you understood and to let them hear how you frame it. **Making space** — leave enough silence for them to reflect; if you jump in reflexively, **count to five in your head before speaking again**.	vs being unhelpful, which is exactly how it feels: when you are new to coaching **it feels very strange not to just help**. The stated reason not to is mechanical rather than moral — **you cannot know every detail of the situation**, so when you supply a solution the coachee is likely to reflexively respond with a **"yes, but…"**, a reason your advice cannot work. Good coaching instead **draws out their own best ideas, gives them space to reflect, and helps them take their own journey to a solution that works for them**. Expect to be bad at it at first; these are odd skills that sound straightforward and take deliberate effort to learn.

Scaling teaching to a group	Teaching one person at a time is great for that person and **a slow way to spread information**. The group version is a class.	Putting one together **takes a ton of effort — a high up-front cost the first time, which you amortise every time you teach it again**. Like individual teaching it needs a specific goal for what students will know or be able to do at the end, and it must **include exercises or some way to practise**. The asynchronous form is a self-paced **guided walkthrough that takes the student step by step through building something or solving exercises**.	vs a talk, which is Part 1's mechanism and aims only at transmission. The worked example is a documented **learning path** for a project surviving on a rotating cast of short-term contributors, many new to the company or the industry: dropping them into an intimidating codebase would have made them run screaming, so **day one** was two trivial changes — adding a joke to a repository of jokes, and adding their own name to the team list — deliberately given a little back-and-forth **so they would get used to the review process**. **Day two** was building and running a tiny purpose-built client and server, watching it in production, looking at its interface, logs and metrics, then making a local change to the library and deploying it **to prove to themselves that the code still worked and that they could see their change in the monitoring data**. By week two, when they made real changes, **they were not scared of the code — it was just code**. With most contributors staying only three months, the same path repaid its investment quickly.
Teaching people to teach	The catalyst tier: scale a class by **handing it to other teachers** so that it has life without you.	Different teachers have different styles — **embrace that**. Let new teachers begin to own their own classes: they should have access to edit the slides and exercises, or your blessing to create their own variants. **Shadow them and give honest feedback: they want to learn.** Once they start teaching other people to teach the class, it has life without you — at which point the right move is to **remove yourself from the teaching rotation**.	vs delegation, which is Part 4's mechanism for work rather than for capability. The other catalyst move is placement rather than creation: if the class applies to all your engineers, **get it into an onboarding curriculum or internal learning path**, so every new hire learns it without you having to find a way to reach them. And where no such culture exists, advocating for an onboarding process or making it easy to create guided walkthroughs is itself the catalytic act.

CONCEPT BOUNDARY — TEACHING IS MEASURED AT THE LEARNER, NEVER AT THE TEACHER

What it sounds like: a more thorough version of advice — you explain more, you explain better, you go through it twice. On that reading, a good session is one where you covered everything clearly.
What it actually is: an attempt to change what somebody can do without you. That single difference is what generates every technique in this part. The goal is stated as a capability the learner has afterwards. The learner has to do the work, because activation is where the learning is. Review comments must carry the *why*, or they only fix today's change. Coaching withholds the answer, because a solution you supplied is a solution they can reject with a "yes, but…" and cannot reuse.
Why that matters practically: it gives you a test at the end of every session that has nothing to do with how well you explained. Can they draw the system and describe it to someone else? Can they send the pull request? Can they handle the three to five common scenarios alone? If not, the session did not work, however good it felt.
The trap: mistaking your own fluency for their understanding — which is most seductive precisely when you are best at the thing. The counter is to hand over the keyboard, ask the open question, and then be quiet for five seconds longer than is comfortable.

Anchor sketch — aiming at understanding rather than transmission

Legend: bold = a named technique or rule **reversed = the fact everything else answers** **Q = cover the page and answer this**

```
   ADVICE AIMS AT TRANSMISSION. TEACHING AIMS AT THE
   OTHER PERSON INTERNALISING IT.
        so the test is never how well you explained.
        it is what they can now do WITHOUT YOU.
        and it is not a seniority thing: any knowledge
        gap in EITHER direction is a chance to teach.
                          |
   SET A GOAL, LIKE A LESSON PLAN
        overview of a system -> they can DRAW IT and
             describe it to someone else
        walk a codebase -> everything they need to send
             their FIRST PULL REQUEST
        a tool or interface -> THREE TO FIVE common
             scenarios they can handle alone
        -> every goal is a CAPABILITY, checkable at the end.
   ACTIVATE THE KNOWLEDGE . they do, not just listen.
        their laptop, their hands, their commands.
        best if they leave with an ARTIFACT -- a diagram,
        a snippet -- that outlives the session.
   ------------------------------------------------------
   THE SPECTRUM OF WORKING TOGETHER . not a ranking
        SHADOWING ---------- PAIRING ---------- REVERSE
        you do it,          both of you,        they do it,
        they watch          together            you watch
        teach by            share knowledge     they LEARN
        DEMONSTRATING       + check for         MOST here.
        + visible role      understanding       also a
        model                IN REAL TIME       GUARDRAIL
        Q which end is a guardrail as well as teaching?
   ------------------------------------------------------
   REVIEW, TO TEACH . can boost confidence -- or destroy
```

it. condescending, arbitrary or baffling comments are
soul-destroying. six rules:
1 UNDERSTAND THE ASSIGNMENT . learning or a safety
     check? first draft -> foundations, no nitpicks.
     last review before launch -> into the weeds, no
     big directional questions.
2 EXPLAIN WHY, NOT JUST WHAT . "use the other pointer
     type" fixes today. they may not RECOGNISE why it
     applies next time. link the specific article,
     not the manual.
3 GIVE AN EXAMPLE OF BETTER . "make this clearer" is
     unusable. offer two guesses at what they meant.
4 BE CLEAR WHAT MATTERS . they cannot calibrate you.
     annotate: SECURITY HOLE / house style + link /
     my preference, your call / just a nitpick.
5 CHOOSE YOUR BATTLES . passes, high level first.
     if it does not do what it should, indentation
     is irrelevant. wrong problem? then not 100
     technical notes.
     -> lots to fix? TALK or PAIR instead of burying
        them in comments.
6 IF YOU MEAN YES, SAY YES . mark blocking vs not.
     say "looks good to me" ON DOCUMENTS -- there is
     no merge button, and with many reviewers each
     waits for the others.
     -> juniors may be reluctant to question you even
        when they think you are wrong.

COACHING . mentoring shares YOUR experience.
coaching teaches them to solve it THEMSELVES.
     OPEN QUESTIONS . not yes/no. dig into the problem
          before any solution.
     ACTIVE LISTENING . reflect it back. your framing
          may hand them a new way to describe it.
     MAKING SPACE . silence. COUNT TO FIVE before you
          speak again.
     why not just answer? you cannot know every detail,
     so a supplied solution earns a "YES, BUT...".
     expect to be bad at this at first.

SCALING IT
GROUP . a CLASS. high up-front cost, amortised on every
     repeat. specific goal + exercises. async version:
     a self-paced guided walkthrough.
     worked example -- a LEARNING PATH for 3-month
     contributors: day 1, two trivial changes (a joke,
     their own name) with deliberate back-and-forth to
     learn REVIEW. day 2, run a toy client and server,
     see its logs and metrics, change the library and
     deploy to see the change in the monitoring.
     by week 2 the code was JUST CODE.
CATALYST . teach other people to TEACH it. give them
     edit rights or your blessing to make variants.
     shadow them, give honest feedback. once they are
     teaching teachers, LEAVE THE ROTATION.
     or place it: get it into the ONBOARDING CURRICULUM
     and every new hire gets it without you.

Q Give the three points on the working-together spectrum and what each one buys.

Q Name the six review-to-teach rules, and the four levels of importance a comment can carry.

Q Give coaching's three skills and the specific reason not to supply the answer.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Teaching rather than advising	Something that survives the conversation — a capability rather than a fact.	Considerably more of your time, and a goal you have to define before you start.	When the gap is a skill rather than a missing piece of information.
Reverse shadowing rather than demonstrating	The densest learning available: they activate the knowledge and practise the task, and you get a guardrail for free.	It is slower, more error-prone in the moment, and hard to watch.	Once they have seen it done. Demonstration first, participation next, then their hands only.
Explaining why in a review comment	The author knows what to do next time, not just this time.	Longer comments, and the work of finding the specific article rather than the general manual.	Whenever the point generalises. A one-off correction can stay a one-off correction.
Reviewing in passes rather than all at once	The directional problem gets found before anyone spends effort on the detail.	More rounds, and a longer wall-clock review.	Whenever the work might be solving the wrong problem. If it needs a lot, talk or pair instead of commenting.
Coaching rather than answering	They make their own connections, which is where the durable learning is — and you avoid a solution built on details you do not have.	It is slower, it feels strange, and you should not expect to be good at it immediately.	When they have the raw material to reach it themselves, and when your answer would attract a “yes, but…”.
Building a class	Every future run is cheap, and the material outlives the sessions.	A high up-front cost the first time, plus exercises, plus maintenance.	When you have taught the same thing individually enough times to know what the goal and the exercises should be.
Teaching other people to teach your class	The class continues without you, in styles you would not have used.	Editing rights, variants you did not choose, and the discipline of shadowing and giving feedback.	When the class is proven. Then place it in an onboarding path, and leave the rotation.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“I walked them through the whole system.”	“Can they draw it on a whiteboard and describe it to someone else? That was the goal.”
“I showed them how to use it.”	“Whose laptop was it on? If they didn't type anything, they watched — they didn't learn.”
“I'll do it and they can watch.”	“That's shadowing, which is the right start. Next time we pair, then I sit on my hands and watch them.”

“I told them to use the other type.”	“Did I say why? Otherwise they'll be back next week with the same choice and no way to make it.”
“I asked them to make the section clearer.”	“That's unusable. Let me offer two versions of what I think they were trying to say.”
“I left forty comments on the design.”	“Is my first comment that they're solving the wrong problem? Then the other thirty-nine are noise — let's talk instead.”
“I had nothing to add so I didn't reply.”	“Silence reads as unresolved. On a document there's no merge button — I need to say it looks good to me.”
“They asked me a question so I answered it.”	“If I'm coaching, I ask an open question instead — I don't have every detail, and a supplied answer gets a 'yes, but'.”
“There was an awkward silence so I filled it.”	“Count to five. The silence is where they do the thinking.”
“I've run this class six times now.”	“Then it's time someone else owned it — with edit rights and my honest feedback, and then I step out of the rotation.”

PART 2 · QUESTION 1

A senior engineer is bringing a new joiner up to speed on a payments ledger. They book two hours, screen-share, and talk through the architecture while the joiner listens; at the end they ask “does that make sense?” and get a yes. Three weeks later the joiner opens their first change and cannot explain the flow to a colleague. Meanwhile the same engineer reviews that change with sixty comments, opening with “I don't think this belongs in this service at all” and continuing through naming, indentation and a suggestion to use a different collection type, with no indication of which comments block the merge. Two other reviewers have not responded, though neither has objections. When the joiner asks the engineer how to decide between two designs, the engineer tells them which one to pick.

(a) State what the two-hour session was missing on two separate counts, and give a goal it should have had. **(b)** Name the three review rules the sixty comments break, and give the reasoning attached to each. **(c)** Explain what the two silent reviewers are doing, why it happens, and what the rule requires of them. **(d)** The engineer answered the design question directly. Name the alternative approach, give its three skills, and state the specific mechanism that makes a supplied answer weaker.

PART 2 · QUESTION 2

A platform team takes on six-month apprentices who arrive knowing little of the stack. The current plan is a two-day lecture series in week one, delivered live by whoever is free, followed by real tickets. Apprentices report being terrified of the codebase for their first month and rarely open changes without asking someone to check first. A staff engineer proposes instead to write a self-paced guided walkthrough and a class, and to run the class personally every intake — she has now run it four times and it takes her three days of preparation each cycle, and two other engineers have asked whether they could teach it but she has told them her slides are the canonical version. The team's onboarding page exists but does not mention any of this.

(a) Name what the lecture series omits, and give two properties any replacement class must have. **(b)** Design the first two days of a learning path for this team, following the structure of the worked example, and say what each day is deliberately teaching. **(c)** The engineer is repeating a three-day cost every intake. Name the tier she is stuck at, what the next tier requires, and the two things she must give the other engineers. **(d)** Give the placement move that would reach every future apprentice without her, and say what she should do once other people are teaching teachers.

2×2 — whose hands are on it, and is there a stated goal

COLUMNS: IS THERE A STATED GOAL — A CAPABILITY THEY WILL HAVE AT THE END? →

	Yes — you can check it when the session ends	**No — you set out to cover the material**
THEY do the work	**Teaching** They type, they draw, they run it, and both of you know what they should be able to do afterwards — so you find out in the room. *Move:* make sure they leave with an artifact, then move along the spectrum until the next session is one you do not attend.	**Busy, and unmeasured** Plenty of activity and no way to tell whether anything transferred. It feels productive because somebody was working throughout. *Move:* pick the goal retroactively and test it — can they draw the system, or send the first change? Whatever fails is the next session.
YOU do the work	**Demonstration** Legitimate and limited: shadowing teaches by showing and models high standards — but the learning stops where their hands would have started. *Move:* a first step, not a method. Name it as shadowing, and say when the pairing session is.	**A talk with an audience of one** You explained it well, they said it made sense, and nothing is different. Advice wearing teaching's clothes. *Move:* ask what they should be able to do afterwards. If you cannot answer that, the session cannot succeed or fail.

Read it as: the row is *activation*, the column is *measurement*, and the bottom-right is the default everyone falls into because it is the cell that feels most like teaching from the inside. “Does that make sense?” is its signature question: answerable only by the person least able to know.

CARRY-OUT RULE FROM THIS PART

Decide what they should be able to do at the end, then arrange for their hands to be the ones doing it — and when you review, carry the *why* and mark what matters, because a comment that fixes today's change and teaches nothing has done half its job at full cost.

3

PART 3 OF 4

Making it safe to move fast

Why a railing is not a gate, the six things a real review looks for, and how a guardrail stops needing you.

TENTH-GRADER VERSION

- Think of the railing on a clifftop path. It isn't there to lean on. It's there so a stumble doesn't kill you.
- Because the railing exists, you can walk faster and look at the view. That's the whole point: guardrails let people be brave.
- Reviewing someone's work is a railing — but only if you actually read it. Waving it through is worse than not reviewing at all, because they thought someone had checked.
- You can also be a railing for a whole project: ask the question that makes them notice they're heading for a cliff.
- For a lot of people at once, you write down the steps, or get a computer to do the checking, or put the reminder where they can't miss it.
- Make the right way the easy way. If the official route is slow and annoying, people will go round it.
- The strongest railing is when everyone simply believes in it. Until then you'll be chasing people forever.

THE EVERYDAY VERSION

A climbing gym does not make climbing safe by telling people to be careful. It bolts anchors into the wall, puts mats on the floor, and trains a second person to hold the rope. Notice what that arrangement actually produces: not caution, but the opposite. People try the route they would never attempt on bare rock, fall off it, and try again — **the safety is what buys the boldness**. Notice also what happens when the arrangement is fake. A person holding the rope while looking at their phone is far more dangerous than nobody holding the rope at all, because the climber has adjusted their behaviour to a protection that is not there. And notice the last thing: the strongest gyms are not the ones with the most rules on the wall. They are the ones where a climber who tied a bad knot gets three people saying something before they leave the ground, without anyone consulting a policy — **because at that point it is not a rule any more, it is simply what people here do**.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
A guardrail	Like the railings along a clifftop path: **not for leaning on, but there to steady yourself when you need them**. A small stumble will not doom you — the railing stops you going over the edge.	The purpose is not restraint but speed: **guardrails encourage autonomy, exploration and innovation**, because **we all move faster when the going is safe**.	vs a rule or a gate. Everything in this part follows from that inversion — the guardrail exists so that someone else can take a risk you would otherwise have to talk them out of. If a supposed guardrail is making people slower or more timid, it is not doing the job the metaphor describes.

Review as a guardrail	The same three reviews that teach — **code, design and change review** — used to prevent harm rather than to impart understanding.	The third is worth naming separately: **change management** is writing down exactly what you are going to do before you do it and having someone else agree you have described the right set of steps — **like code review, but for command lines or clicking buttons in the right order**. The effect on the author is the point: **when someone knows their work will be reviewed, it is easier for them to feel confident working independently**, because they know a check exists that will stop them causing an outage or spending months on an architecture that cannot work.	vs review as teaching, which is Part 2 and aims at the author's understanding. Here the aim is the system, and there is one absolute instruction: **if you want to be a good guardrail, never rubber-stamp changes** — read carefully, every line of code, every section of a design, every step of a proposed change. A rubber stamp is worse than no review, because the author has already adjusted their care to a check that did not happen.
Category 1 — should this work exist?	What problem does your colleague intend to solve?	In particular: **are they using a technical solution to solve a problem that should have been solved by talking to someone?**	vs whether the work is good, which is every other category. This one asks whether the correct answer is no code at all — and it has to be asked first, because nothing further matters if it is.
Category 2 — does it actually solve the problem?	Will the solution work?	Will users be able to do what they need and **what they expect**? Are there errors or typos, bugs or performance issues? **Does the design propose using a system in a way that will not work?**	vs correctness in the narrow sense. Note the two-part user test — what they need *and* what they expect — and that the last clause is about somebody else's system, which the author may have assumed capabilities of without checking.
Category 3 — how will it handle failure?	What happens when things go wrong, which they will.	Weird edge cases, malformed input, **the network randomly disappearing**, load spikes, or whatever else can happen. **Will it fail in a clean way, or will it corrupt data or take a user's money without giving them the service they have paid for?** And: how will you discover problems?	vs testing, which the author has probably done for the paths they imagined. The framing that makes this category productive is the second sentence: not *does it fail* but *what kind of failure is it* — and the money example is the one to keep, because it is the class of failure that is silent and irreversible.
Category 4 — is it understandable?	Will other people be able to maintain and debug it?	Are the components or variables **named intuitively**? **Is the complexity contained in a well-chosen place?**	vs readability as a style question. The second clause is the load-bearing one and it is a design property rather than a formatting one: complexity that has to exist should sit somewhere deliberate, so that the rest of the system can be reasoned about without it.
Category 5 — does it fit into the bigger picture?	The effect of this change on everything that is not this change.	**Does it set a precedent or create a pattern you might not want other people to copy?** Does it force other teams to do extra work for future changes? **Is this a risky change scheduled at the same time as a high-profile launch?**	vs scope creep in review. This is the category the author is structurally least able to check, because all three questions are about context outside their own work — which is exactly why a reviewer with wider scope is the right person to ask them.

Category 6 — do the right people know about it?	Whether the humans who need to act have been told, and know what is expected of them.	Is everyone copied on the change who should be? **Are there names attached to any actions that need to happen, or is there a lot of passive voice where it is not clear which team is doing what?** Do the people involved know what is expected of them?	vs communication as an afterthought. The passive-voice test is a concrete, checkable thing to look for in a document: an action with no named actor is an action nobody has agreed to do.
The reviewer's stance	How to hold all six without becoming the obstacle.	**Be open to the idea that you do not know everything.** Ask questions and be constructive.	vs gatekeeping, and this is the sentence to keep: **a good guardrail is not an arbitrary gatekeeper — you are on the same side as the person you are keeping safe, and you want them to succeed**. It is the difference between a railing and a fence, and it is audible in the first comment you leave.
Project guardrails	Being the more experienced presence beside somebody who is stretching: **they will let you know if you are getting close to a disaster you will not be able to recover from**.	It is not doing the work for them or protecting them from all possible mistakes. The worked form is a weekly question — **"just out of interest, how were you planning to balance these two conflicting business priorities?"** — asked of somebody who had not noticed the problem creeping up, and which was enough to put them back on course. The guardrail was making sure they had noticed they were **walking close to a cliff edge**, and standing ready to coach if they had no idea what to do.	vs mentoring for juniors. **It is not just for less experienced folks**: you can play this role for any colleague leading a project or taking on a difficult task, and **even very seasoned people can use support in a project that uses a new skill set**. Note the quiet signal too — **if someone asks you to be a mentor or adviser on a project, they are probably hoping for at least a little guarding**. And a guardrail can offer **support as well as safety**: be specific about how you can help — promising to review designs, or to advocate for ideas with upper management — and explicit about **when and how they should ask**, as in "email me if that person hasn't replied to you in three days; I can be your muscle there".
Processes	The group-tier guardrail: rather than individually teaching your coworkers the right thing to do, **write down a standard set of steps and convince the organisation to follow them**.	A prelaunch process might answer: do we need security approval? how much notice should we give marketing or customer support? should we ship behind a feature flag? is there standard monitoring, eventing and documentation to add? do we need to tell other teams to expect extra load? Other places a process or checklist helps: **responding to a major outage or security incident; sharing and agreeing on designs; adopting a new technology or language; and making and recording decisions that cross multiple organisations**.	vs bureaucracy, and the honest framing is that **opinions vary and nobody is wrong — it is a trade-off**. Some people are delighted by clear steps and the safety of standardisation; others insist that people should think rather than follow protocols, and that checklists and approvals just slow them down. What settles it is scale: **the bigger the company, the more likely you need some structure that helps people do the right thing without asking the same questions every time** — because as the organisation grows there are more ways for a launch to cause an outage or a public-relations mess, **and so a process is born**. The design rule: **aim to make processes as lightweight as possible**, since a complicated procedure with boilerplate, central approval and long waiting periods is not a good guardrail — **people will just sneak around it**. **Make the right way the easy way.**

Guidelines are wrong sometimes	The three-way bind that any written process faces, and the honest way out of it.	**Write nothing down** and most people hate it and complain they do not know how to do anything. **Write guidelines** and people interpret them as law and argue they are wrong because they do not cover the edge cases. **Write down every edge case** and you end up with a binder of policy and legalese that still will not cover every situation — **and everyone still hates it**.	The resolution is stated rather than solved, and it belongs at the top of the document: these answers **will not apply perfectly in every situation; think twice before discarding them, but if they do not make sense for the situation you are in, do the thing that makes sense instead. All guidelines are wrong sometimes** — and if they are wrong a lot, propose a change. Underneath it sits the general instruction: **think hard about the other humans involved, assume they are reasonable people trying their best, and be a reasonable person trying your best**.
Written decisions	Making a decision once and writing it down, **so people do not have to have the same argument again and again**. It removes a little decision fatigue: the rules say we usually do this, so that is what we do.	Four kinds. **Style guides** — a set of conventions, sometimes arbitrary, about how to write code for a project, covering everything from naming conventions to error handling to which language features are acceptable; deciding once saves every new project from the same naming argument and produces more consistent code. **Paved roads** — a documented set of standard, well-supported technologies that teams are recommended or required not to step off, often published with each technology marked by its standing, from “adopt” through to “hold”. **Policies** — enforced rules, such as running a retrospective after every outage. **Technical vision or strategy** — a clear direction within which teams still choose their own paths.	The four are not equivalent and the warning lands on one of them: **use policies sparingly**. It is hard to account for all the edge cases and there will always be edge cases — **and if there are too many policies, people just will not remember all the things they are supposed to do**. Note the asymmetry: a style guide removes an argument, a policy adds an obligation, and a vision constrains direction while leaving the route free. The stated stake for a policy is real, too: if the rule is enforced, breaking it could be seen as failing to do one's job, **with implications for performance reviews**.

Robots and reminders	Making it as easy as possible for people to remember to do the right thing — **putting the right solution in their faces, sometimes literally**.	The illustration: a workplace where everyone was supposed to fill in timesheets and everyone forgot, until somebody stuck a timesheet grid and a pen **on the exit door at head height** — push the door to leave and the timesheet is in front of your eyes. The software forms: **automated reminders** that put it in someone's calendar or message them, **including a link to the process**; **linters** that enforce as much of the style guide as they can **so reviewers do not have to**; **search**, arranged so that looking up how to do something surfaces the right way — even if that means updating the wrong documents with headers pointing elsewhere; **templates**, so that if every design document needs a security section, the template has one; and **configuration checkers and presubmits** that run tests or safety checks before a change lands.	vs automation for its own sake. The distinguishing property is that these move the remembering out of the human and into the environment, which is why they scale where reminders-by-person do not. The presubmit example is worth keeping because it shows what a machine-enforced guardrail actually looks like: **no more than 5% of servers may be rebooted at once**, and **do not decommission a server for this system if the on-call for it recently got paged**. And better than any reminder — **have an automated system do the right thing so humans do not have to**.
Culture change	The catalyst tier, and **the most effective guardrail as well as the most difficult to put in place**.	Robots, policies and processes scale further than being a guardrail for individuals, **but they all still rely on you doing something**. For a message to really stick you need everyone to believe in it and care about it — you want the organisation to reach a state where **it would be considered weird to do something else**. The historical argument: most technology companies now do code review and write tests, **but that was not always the case** — all the guardrails we take for granted were introduced by people who cared enough to argue why the change was worth the time.	vs enforcement. The line that separates them: **unless you can make the guardrail part of your culture, you will always be chasing compliance**. Be patient — it takes a lot of time and dedicated effort to make everyone behave differently, and it is **the only way you will ever be able to stop pushing the process along manually**. Four things make the journey easier. **Solve a real problem** — align it with what the organisation needs, and expect a lot of "why" questions, so have answers that are not merely aspirational: what does the business actually get out of this? **Choose your battles** — rather than a process for design review, instil a *respect* for design review and trust teams to choose the form; offer easy defaults and do not get hung up on everyone following exactly the same one. **Offer support** — your processes and automations should make it easy to do the right thing. **Find allies** — do not try to change a culture on your own, and ideally include high-level sponsors and influencers in the organisation you are trying to change.

CONCEPT BOUNDARY — A GUARDRAIL IS PERMISSION, NOT RESTRICTION

What it sounds like: the polite word for control. Reviews, processes, checklists, linters and policies are all things that stand between a person and shipping, so the natural reading is that a guardrail is a brake with a friendlier name.
What it actually is: the thing that makes speed affordable. The railing is not there to slow you down on the clifftop path; it is there so that you can walk it at all without measuring every step. Reviews work the same way — the value is not only the defect caught, it is that *the author can work independently*, because they know a check exists. Take the check away and you do not get a faster engineer, you get a more cautious one.
Why that matters practically: it gives you a test for whether any given guardrail is working, and the test is not compliance. Are people moving faster and attempting more than they would without it? If a process has made everyone slower and more timid, it has failed on its own terms no matter how well it is being followed — and if it is heavy enough that people route around it, it has failed twice, because they are now moving fast with no railing at all. Hence: **make the right way the easy way**.
The trap: the rubber stamp. It is the one failure that is strictly worse than doing nothing, because the author has already spent the confidence the guardrail bought them. If you are going to be the railing, read every line — and if you are not going to, say so, so that somebody else can be.

Anchor sketch — the railing, and how it stops needing you

Legend: bold = a named category or technique **reversed = the fact everything else answers** **Q = cover the page and answer this**

A GUARDRAIL IS NOT FOR LEANING ON. IT IS THERE SO A STUMBLE DOES NOT DOOM YOU -- WHICH IS WHY IT MAKES PEOPLE FASTER, NOT SLOWER.

autonomy, exploration, innovation. we all move faster when the going is safe.

|

INDIVIDUAL . **REVIEW AS A GUARDRAIL**

code . design . CHANGE MANAGEMENT (write down what you will do BEFORE you do it, and have someone agree the steps -- code review for command lines)
the effect on the author: knowing a check exists makes it EASIER TO WORK INDEPENDENTLY.
NEVER RUBBER-STAMP. every line, every section, every step. a fake railing is worse than none -- they already spent the confidence it bought.

SIX CATEGORIES TO LOOK FOR

1 SHOULD THIS WORK EXIST? is it a technical fix for something that needed a CONVERSATION?
2 DOES IT SOLVE THE PROBLEM? what users need AND expect. does it use another system in a way that will not work?
3 HOW WILL IT HANDLE FAILURE? edge cases, bad input, the network vanishing, load spikes. clean failure -- or corrupt data, or take the money and give nothing? how will you FIND OUT?
4 IS IT UNDERSTANDABLE? intuitive names. is the complexity in a WELL-CHOSEN PLACE?
5 DOES IT FIT THE BIGGER PICTURE? a precedent others copy? work forced on other teams? risky, and next to a high-profile launch?
6 DO THE RIGHT PEOPLE KNOW? names attached to

actions -- or PASSIVE VOICE with no actor?
-> stance: be open to not knowing everything.
A GOOD GUARDRAIL IS NOT AN ARBITRARY GATEKEEPER.
you are ON THE SAME SIDE.

PROJECT GUARDRAIL . not doing the work, not preventing all mistakes -- telling them when they are near a disaster they CANNOT RECOVER FROM.
"just out of interest, how were you planning to balance these two conflicting priorities?"
-> not only for juniors. seasoned people using a NEW SKILL SET need it too.
-> asked to "advise" on a project? they are probably hoping for a little guarding.
-> support AS WELL AS safety: name what you will do, and WHEN to call you. "email me if they have not replied in three days."

GROUP . codify it

PROCESSES . write the steps down and get the org to follow them. prelaunch, incidents, design sign-off, adopting a technology, cross-org decisions.
opinions differ and NOBODY IS WRONG -- it is a trade-off. but the bigger the company, the more you need it.
-> KEEP THEM LIGHTWEIGHT. boilerplate + central approval + long waits => people SNEAK AROUND IT.
MAKE THE RIGHT WAY THE EASY WAY.
-> the bind: write nothing (hated) / write guidelines (read as law, argued at the edges) / write every edge case (a binder, still hated).
say it out loud: ALL GUIDELINES ARE WRONG SOMETIMES. if they are wrong a lot, change them.

WRITTEN DECISIONS . decide once, stop re-arguing
STYLE GUIDES conventions, sometimes arbitrary.
PAVED ROADS supported technologies, each marked with its standing from "adopt" to "hold".
POLICIES enforced rules -- USE SPARINGLY. edge cases always exist, and too many policies are simply not remembered.
VISION / STRATEGY direction, route left free.

ROBOTS AND REMINDERS . put it IN THEIR FACES.
the timesheet taped to the EXIT DOOR at head height.
automated reminders (with the link) . linters (so reviewers need not) . search that surfaces the right way . templates that carry the required sections . presubmits and config checkers -- "no more than 5% of servers rebooted at once".
-> best of all: have the machine DO the right thing.

CATALYST . **CULTURE CHANGE**

robots and processes still rely on YOU doing something. for a message to stick, everyone has to believe it -- until doing otherwise would be WEIRD.
code review and tests were once arguments somebody had to win.
UNLESS THE GUARDRAIL BECOMES CULTURE, YOU WILL ALWAYS BE CHASING COMPLIANCE.
four things that help:
SOLVE A REAL PROBLEM . expect "why?" -- have a

```
        business answer, not an aspiration.
CHOOSE YOUR BATTLES . instil RESPECT for design
        review rather than one mandated process.
OFFER SUPPORT . make the right thing easy.
FIND ALLIES . sponsors and influencers. not alone.
```

Q Name the six categories a guardrail review looks for, in order.

Q Why is a rubber stamp worse than no review at all?

Q Give the four kinds of written decision, and the one that carries a warning.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Reviewing every line rather than skimming	A railing that is actually there, and an author who can work independently because of it.	Real reading time, on work you did not write.	Whenever you accept the review. If you cannot do it properly, say so and let somebody else be the railing.
Being a project guardrail	Somebody can attempt a project beyond their current reach without risking an unrecoverable failure.	Standing attention on work you are not doing, and the discipline of asking rather than taking over.	When a colleague is stretching — including a seasoned one using a new skill set, and anyone who asked you to advise.
Writing a process down	People stop asking the same questions, and the organisation stops depending on who happens to know.	Judgement replaced by steps, and a document that will be wrong at the edges.	As the organisation grows and the ways to cause an outage multiply. Keep it as lightweight as you can.
Making it a policy rather than a guideline	Enforcement, and a rule that carries real consequences.	Edge cases it cannot cover, and the certainty that people will not remember many of them.	Sparingly. Prefer a style guide to remove an argument, or a vision to set direction.
Automating the check instead of reminding people	The remembering moves out of the human and into the environment, where it does not decay.	Building and maintaining the automation, and the risk of efficient machinery doing the wrong thing efficiently.	Whenever the right behaviour is mechanical. Better still, have the system simply do the right thing.
Pursuing culture change	The only guardrail you stop having to push, and the state in which doing otherwise would seem weird.	A lot of time and dedicated effort, and patience while nothing appears to happen.	When you are tired of chasing compliance — and only with allies, a real business answer, and support that makes the right thing easy.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“All these reviews are slowing us down.”	“The railing is what lets people move fast. Take it away and you don't get speed, you get caution.”

“It looked fine, so I approved it.”	“Then I was a fake railing. They spent the confidence my review was supposed to buy them.”
“The code works, so the review is done.”	“Should this work exist at all? Is it a technical fix for something that needed a conversation?”
“The design says the data will be archived.”	“By whom? That's passive voice with no actor — nobody has agreed to do it.”
“They're senior, they don't need support.”	“They're using a skill set that's new to them. That's exactly when a project guardrail helps.”
“Everyone keeps skipping the launch checklist.”	“Then it's too heavy. If people sneak around a process, the process is the problem — make the right way the easy way.”
“We'll write a policy for it.”	“Use policies sparingly. Would a style guide or a stated direction settle it without adding a rule nobody remembers?”
“I'll remind them each week.”	“Put it where they can't miss it, or have a machine do it. Reminding by hand doesn't scale and it depends on me.”
“People still aren't following the process.”	“Until it's culture, I'll be chasing compliance forever. What does the business get from it, and who are my allies?”

PART 3 · QUESTION 1

A payments team's design document is circulated for review. It proposes a new internal service to reconcile two teams' conflicting figures, a disagreement that has been running in a chat channel for a month. The reviewer, short of time, reads the first page and approves it with “looks sensible”. The document says that failed reconciliations “will be retried and the discrepancies will be reported”; that the service will call another team's export endpoint every minute; and that launch is scheduled for the same week as the company's largest product launch of the year. Three months later a partial failure silently debits customers without completing their transfers, nobody notices for a day, and the other team says their endpoint was never meant to be called at that rate.

(a) Name the failure the reviewer committed and explain why it is worse than not reviewing at all. **(b)** Work through the six categories a guardrail review looks for, naming for each the specific thing in this document it would have caught, or stating that it would have caught nothing. **(c)** Rewrite the quoted failure-handling sentence so that it passes the category concerned with people, and name the test you applied. **(d)** Give the stance a reviewer is supposed to hold, and say what distinguishes it from gatekeeping.

PART 3 · QUESTION 2

After two bad releases, an engineering director introduces a launch procedure: a fourteen-page document, a mandatory review board that meets on Thursdays, and a rule that any team launching without approval will have it noted in performance reviews. Six months on, launches take three weeks longer, two teams have started describing their releases as “configuration updates” to avoid the board, and the staff engineer who wrote the document spends a day a week chasing people to follow it. Separately, the same organisation has a style guide nobody reads, no automated checks of any kind, and an onboarding page whose search results still point at a procedure retired last year. The director's proposal is to add a second policy requiring teams to confirm in writing that they read the first one.

(a) Name the three design faults in the launch procedure and give the rule each one breaks. **(b)** Explain what the “configuration updates” renaming demonstrates, and state the principle that predicts it. **(c)** The staff engineer is spending a day a week on this. Name what they are doing, what it means they have not achieved, and the four things that make the alternative easier. **(d)** Take the style guide, the missing automation and the stale search results together: name three specific mechanisms that would put the right behaviour in people's way, and say which of them removes work from reviewers.

2×2 — is the right way the easy way, and does it survive you

COLUMNS: DOES IT KEEP WORKING WHEN YOU STOP PUSHING? →

	Yes — automated, or believed in	**No — it runs on your attention**
The right way is the EASY way	**A real guardrail** The template carries the section, the linter catches the style, the reminder arrives with the link — or people believe in it, and doing otherwise would seem weird. *Move:* keep it light. The moment the easy way stops being easy, you are back to chasing.	**Working, on your battery** People do the right thing because you keep asking. Reviews get read because you read them; the checklist is followed because you chase it. *Move:* convert the mechanical parts to automation and the human parts to belief.
The right way is SLOW or awkward	**Durable and routed around** The board meets and the rule has teeth — and the work has been quietly renamed so it no longer qualifies. The guardrail survives; the safety does not. *Move:* take weight out rather than adding enforcement. Boilerplate, central approval and long waits produce the workaround.	**Chasing compliance** Heavy, unloved and dependent on you. Every week costs attention and buys less than the week before. *Move:* stop adding rules — a policy about the first policy digs this cell deeper. Pick the behaviour that matters and make it the easy path.

Read it as: the row is *friction*, the column is *durability*. The bottom-left fools people: documented, mandatory, enforced — and nothing has gone wrong lately because nothing is going through it.

CARRY-OUT RULE FROM THIS PART

Judge a guardrail by whether people move faster with it, not by whether they comply with it — a process heavy enough to route around has moved the risk rather than removing it.

4

PART 4 OF 4

Handing over the work

The mechanism that actually develops people, what makes a handover real, and how to spend influence on somebody else.

TENTH-GRADER VERSION

- People learn by doing — more than from being told, shown or coached. So the most powerful thing you can give someone is the work itself.
- Don't tidy the project up first. A neat, fully-scoped job teaches almost nothing; a messy one with a safety net teaches a lot.
- Pick someone who'll find it a stretch. If they could already do it perfectly, they won't learn anything.
- They won't do it your way. Let them, as long as they'll get there.
- Here's the tell that you didn't really hand it over: someone asks you about the project and you answer. Send them to the new owner instead.
- Sponsorship is spending your own reputation to get someone an opportunity. It's more work than advice and worth much more.
- Watch who you pick. It's very easy to keep picking people who remind you of yourself.
- And stop being the one who answers first in meetings. Leave the gap.

THE EVERYDAY VERSION

Someone has run the same community event for six years and wants to hand it on. There are two ways to do it. The first: they plan the whole event, book the hall, agree the programme, then ask a newer volunteer to “run it” on the day. The volunteer does, competently, and learns approximately nothing, because every decision that mattered was already made. The second: they hand over a half-formed idea, a budget, a list of people who might help, and a standing promise — *call me if the hall falls through*. The event is worse than it would have been. It is also the first thing that volunteer has ever owned, and next year it will be better than either of them could have made it. Now watch the six months after the handover, because that is where the giveaway actually succeeds or fails. Someone emails the old organiser to ask about the date. They know the answer. If they reply, they have quietly taken the event back — the asker will email them again next time, and the new organiser has been made into an assistant without anybody deciding to do it. **The correct answer is the two-line one that says: she's running it now, here's her address, she'll know better than I do.**

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH

Opportunity	Finding people the experiences they need to grow. It is the strongest of the four mechanisms because **people learn by doing, even more than they do from teaching or coaching or advice**.	Every project or role is a chance for **visibility, relationships, and résumé lines — all of which can lead to further opportunities**. That compounding is the reason a small opportunity is worth more than it looks.	vs reward, which is how opportunities are often allocated — to whoever has already proved they can do it. That instinct is precisely backwards here, for the reason in the delegation rows below. Also note the two directions: you can hand out experiences you own, and you can point people at experiences you do not.
Delegation	Giving part of your work to someone else. You are **usually not just tossing someone a project and walking away — you are invested in the outcome**.	That investment is exactly what creates the temptation: **to micromanage, or to handle all of the difficult parts of the project yourself**. The instruction is blunt — **when you hand over the work, really hand it over**.	vs assigning a task. The distinguishing property is ownership, not workload: a task you are still steering has been shared, not delegated, and the person doing it is learning execution rather than judgement.
The wrapped gift versus the messy project	What you hand over determines what they learn.	Colleagues **will not learn as much if you only delegate work after you have turned it into beautifully packaged, cleanly wrapped gifts**. Give them instead **a messy, unscoped project with a bit of a safety net**, and they get a chance to **hone their problem-solving abilities, build their own support system, and stretch their skill set**.	vs setting someone up to fail. The safety net is not optional in that phrasing — the messiness is the lesson and the net is what makes it survivable. **A messy project is a learning opportunity that is hard to get otherwise**, which is the argument for deliberately not tidying it first.
Choosing who	Targeting the difficulty to the person, and looking further than the obvious candidate.	**Do not throw organisational chaos at a new graduate** — but when you look for someone to delegate to, **think beyond the most obvious people**. The decisive line: **anyone who can do an A+ perfect job on a project is not going to learn from it**. Look instead for **someone who will find the work a bit of a stretch but manageable with support — and promise them that support**.	vs picking the safest pair of hands, which optimises this project at the cost of every future one. Two further moves belong here. They **may need a little push to see that the work is within their reach**: they might not yet see themselves as a project lead or an incident commander, and **the fact that you see them that way can be a tremendous boost to their confidence**. And the offer should be concrete — **describe the guardrails you can provide and explain why you think they can handle it**.
You are not going to get a clone	The person you delegated to **will inevitably take a different approach than you would have**.	The response is not to correct it: **be a guardrail, coach them, and ask questions, but inhibit the urge to step in**. **So long as they are going to achieve the goals, let them do it their way.**	vs quality control. The relevant image is **giving away your Legos**: there is natural anxiety that the new person will not build your tower the right way, that they will take all the fun or important pieces, or that if they take over the part you were building there will be none left for you — and **giving away responsibility is the only way to move on to building bigger and better things**.

Do not proxy the questions	The behaviour that silently reverses a handover: **answering questions about the project instead of redirecting them to the new owner**.	You may think you have the current state, but **answering makes you the point of contact, potentially undermining the colleague you handed the project to** — and **you might also have the wrong answer**. Instead, give visibility to the person who took it over: note that they are the expert and owner for the topic, and **show that you trust them to make decisions**.	vs being helpful, which is exactly what it feels like — the question was quick and you knew the answer. Three things follow from redirecting: the project owner **gains a connection to someone new** and whatever comes out of it, **the next time that person has a question they will know where to go**, and **the project is off your plate**. This is the single most checkable test of whether a delegation was real.
Sponsorship	**Using your position of influence to advocate for someone else.** It is **more active than mentorship**: you are deliberately unlocking opportunity rather than giving advice.	It takes more work too — to be a great sponsor **you need to know what your colleague will benefit from and what opportunities they are looking out for**. And it is not free to you: **you are investing your time and social capital in their growth**.	vs mentorship, which shares your experience and costs you an hour. Sponsorship spends something you cannot get back quickly, which is why the choosing question below matters so much.
The four parts of sponsorship	What advocating for someone actually consists of.	**Amplifying** — promoting your colleague's good work and making sure other people know about their accomplishments. **Boosting** — recommending them for opportunities and endorsing their skills. **Connecting** — bringing them into a network, giving them access to people they would not otherwise be able to meet. **Defending** — standing up for them when they are unfairly criticised, and changing negative perceptions of them.	vs praise, which is only the first of the four. The compounding argument attaches here: the opportunities may seem small and **they lead to greater things — recommend someone to lead a small project and you are setting them up to later be seen and chosen for a bigger one**. Every shout-out, comment or reshare signal-boosts their work so that others think of them when opportunities arise; if someone in another team is a superstar, **make sure their boss knows**; and where written peer reviews exist, **a review is a great way to make sure the good work goes on the record**.
Who to sponsor	Look for **people who want opportunities to grow and who you trust to do good work**.	Because **you are spending your social capital by recommending them**, do not waste it on people who do not actually want the opportunity or who will not put in the effort. Sponsor colleagues **who have untapped potential, who are worth investing in**.	The named hazard is **in-group favouritism** — a cognitive bias that **can let you evaluate other people more favourably if they are like you**. The observation quoted against it is that an industry which talks about being a meritocracy is closer to a **"mirrortocracy"**, since people tend to hire people who look like themselves at greater rates than other sectors do. So **pay attention to who you are recommending or helping, and make sure you are not accidentally only sponsoring people who look like you — it is surprisingly easy to do**.

Connecting people to information	Offering opportunities that are not yours to give, **just by knowing that they exist**.	At this level you will probably have **broader context than other engineers you work with**, because you spend more of your time simply knowing things. So keep an ear out: a conference's call for papers is open, a team lead role is opening up, a new internal training programme offers a skill somebody is trying to learn.	vs networking. The stated effect is precise and worth keeping: **by connecting people with information, you expand their options**. It costs you almost nothing, requires no authority, and works even where you have no project to delegate and no standing to recommend.
Sharing the spotlight	The group tier: as the most senior person there will be many things you do best, which makes it **tempting to jump on every difficult problem**.	But while it may feel amazing to be the resourceful, knowledgeable senior person leading the team to greatness every day, **what you are actually doing is overshadowing the rest of the team and preventing them from growing**. Instead, translate your superstardom into helping everyone do better work: **let other people do work that is not as good as you would have done it, so long as it is good enough — that is how they learn**. Three concrete moves: if someone asks a question in a group meeting, **leave a gap or explicitly hand off the floor** to another person on your team; **add a less senior colleague to a meeting you go to, and let them speak about their work**; and **invite a less senior colleague to review designs or code that connects to their work, and be clear that their opinion matters**.	vs delegation, which hands over a defined piece of work. Sharing the stage **includes delegation but also means making space** — letting other people notice that the work needs doing, so they can build leadership skills by picking it up or by learning to delegate it themselves. And it has an opposite failure with its own warning: **if you prefer to delegate everything, make sure you are aligned with your management chain about how you work**, because **most managers will expect some direct execution and visible accomplishments — do not put yourself into so much of a support role that nobody is quite sure what you do**.
Keeping opportunity flowing	The catalyst tier: making sure opportunity and sponsorship continue when you are not there.	Most workplaces have some concept of mentorship, but **sponsorship might not already be well understood**. Look for ways to teach colleagues **to look beyond the obvious candidates** to suggest for opportunities — by advocating for an inclusive interview process, inviting a speaker on implicit bias, or **setting the culture that open team roles are posted on an internal job board**.	vs running a programme. The widest version is stated as a chain rather than a structure: **the best way to be a catalyst in the industry is by empowering your colleagues to become great engineers who do great things** — as you offer the opportunities that turn midlevel engineers into seniors and then into staff engineers, **teach them to offer advice, teaching, guardrails and opportunities to the people who follow them**. Then **the staff engineers you grew will grow the staff engineers who follow them, and your leadership will keep going**.
The bar you are measuring against	A correction for a specific recurring complaint about the people arriving at your level.	**For someone to be promoted to your level, they do not need to be as good as you are now: they need to be as good as you were when you were first promoted to the level.**	vs falling standards, which is the conclusion people reach. The instruction is to reach the other one: **if you keep seeing people join your level who are not as capable as you are, do not snark about lowered standards — think about whether that means you have grown**.

CONCEPT BOUNDARY — A HANDOVER IS MEASURED SIX MONTHS LATER, BY WHO GETS ASKED

What it sounds like: an event. You decide to delegate, you tell the person, you tell the team, and the thing is now theirs. On that reading, a handover succeeds or fails at the moment it happens.
What it actually is: a change in where a stream of small decisions lands, and every one of those decisions is a chance to quietly take it back. The temptations are all reasonable in isolation: handling the difficult part yourself because you are faster at it; stepping in when they choose an approach you would not have; answering a question about the project because you happen to know the answer. None of those looks like reneging. Together they produce a person nominally running something whom nobody consults.
Why that matters practically: it gives you a test that does not depend on your intentions. When somebody asks *you* about the project, what happens? If you answer, you are the owner and they are your assistant, whatever the announcement said. The correct move is to name them as the expert and owner, show you trust them to decide, and let the asker build the relationship — which also hands them the connection and everything that comes from it.
The trap: tidying the work before you give it away. A cleanly wrapped, fully scoped project protects the outcome and removes the lesson, because every decision that would have taught them something has already been made. Hand over the mess and hold the net instead.

Anchor sketch — giving work away, and making it stay given

Legend: bold = a named practice or rule **reversed = the fact everything else answers** **Q = cover the page and answer this**

PEOPLE LEARN BY DOING, EVEN MORE THAN FROM TEACHING OR COACHING OR ADVICE.

so opportunity is the strongest of the four --
and every project is visibility, relationships and
resume lines, WHICH LEAD TO MORE OPPORTUNITIES.
|

DELEGATION . you are invested in the outcome, which is exactly why you will want to micromanage or keep the hard parts. WHEN YOU HAND IT OVER, REALLY HAND IT OVER.

- WHAT you hand over decides what they learn:
  - a BEAUTIFULLY PACKAGED, CLEANLY WRAPPED GIFT
    -> teaches almost nothing
  - a MESSY, UNSCOPED PROJECT WITH A BIT OF A SAFETY NET -> problem-solving, their own support system, a stretched skill set
- WHO . target the difficulty. no organisational chaos for a new grad -- but look BEYOND the obvious people.
  ANYONE WHO CAN DO AN A+ JOB WILL NOT LEARN FROM IT.
  want: a stretch, MANAGEABLE WITH SUPPORT -- and promise the support.
  they may not see themselves as a lead yet. THAT YOU DO can be a tremendous boost. name the guardrails and say why you think they can.
- YOU WILL NOT GET A CLONE . they will do it differently. be a guardrail, coach, ask questions, and INHIBIT THE URGE TO STEP IN. goals met their way is a success.
  -> GIVING AWAY YOUR LEGOS: fear that they build the tower wrong, take the fun pieces, leave none for you. giving it away is the ONLY way

to build bigger things.
THE TELL: WHO GETS ASKED? answering a question about the project MAKES YOU THE POINT OF CONTACT, undermines them -- and you may have the WRONG ANSWER. redirect: they are the owner, and I trust them to decide.
-> they gain the connection . the asker learns where to go . and it is OFF YOUR PLATE.

SPONSORSHIP . using your position to ADVOCATE. more active than mentoring, and it costs TIME AND SOCIAL CAPITAL, not an hour.

AMPLIFYING	make their work known
BOOSTING	recommend them, endorse the skills
CONNECTING	bring them into a network
DEFENDING	stand up for them when unfairly criticised; change the perception

small opportunities COMPOUND: lead a small project -> be seen for a bigger one. tell their boss. put it on the record in a written review.
WHO . they must WANT it and be trusted to do good work -- you are spending capital. untapped potential, worth investing in.
IN-GROUP FAVOURITISM . we rate people more highly when they are LIKE US. a "meritocracy" is closer to a MIRRORTOCRACY. check who you keep picking. IT IS SURPRISINGLY EASY TO DO.

CONNECTING PEOPLE TO INFORMATION . you know more things. a call for papers, a lead role opening, a training programme. BY CONNECTING PEOPLE WITH INFORMATION YOU EXPAND THEIR OPTIONS.

GROUP . **SHARE THE SPOTLIGHT**
being the one who solves everything feels great and OVERSHADOWS THE TEAM, preventing them growing. let others do work that is NOT AS GOOD AS YOURS, so long as it is GOOD ENOUGH. that is how they learn.
three moves: leave a gap or hand off the floor . bring a junior colleague to your meeting AND LET THEM SPEAK . invite them to review, and say their opinion matters.
also MAKING SPACE -- let others notice the work needs doing and pick it up.
Q and the opposite extreme?
-> delegate EVERYTHING and nobody knows what you do. managers expect visible accomplishments. align with your chain.

CATALYST . keep it flowing without you
sponsorship is often NOT well understood. teach people to look past the obvious candidates: inclusive interviewing . a speaker on implicit bias . roles posted on an internal job board.
the widest version is a CHAIN: grow seniors, teach them to give advice, teaching, guardrails and opportunity in turn. THE STAFF ENGINEERS YOU GREW WILL GROW THE ONES WHO FOLLOW THEM.

THE BAR . to reach your level they need to be as good as YOU WERE WHEN YOU GOT HERE -- not as good as you

```
are now. if the new arrivals seem weaker, consider
that YOU HAVE GROWN.
```

Q What should you hand over, who to, and what disqualifies the obvious candidate?

Q Give the four parts of sponsorship, and the bias to check yourself against.

Q Name the behaviour that silently reverses a delegation, and the three things redirecting buys.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Delegating the messy version rather than the tidy one	Problem-solving, a support system they built themselves, and a genuinely stretched skill set.	A worse-run project in the short term, and your nerve while it happens.	When you can supply the safety net. The mess is the lesson; without the net it is just a fall.
Choosing someone who will find it a stretch	Someone learns. The obvious candidate would have delivered and gained nothing.	Higher risk on this project, and the support you have now promised.	Whenever the work is survivable if it wobbles. Not for organisational chaos handed to a new graduate.
Letting them do it their way	Ownership that is real, and an approach you would not have thought of.	Watching a different route being taken and saying nothing.	So long as they are going to achieve the goals. Be a guardrail and ask questions instead of stepping in.
Redirecting a question you could have answered	Their authority in front of the asker, a new connection for them, and the project genuinely off your plate.	Two minutes of extra friction for the asker, and giving up being the person who knew.	Every time. This is the behaviour that decides whether the handover was real.
Sponsoring somebody	Opportunities unlocked that advice could never have produced, and a compounding effect as small chances lead to larger ones.	Your time and your social capital, which do not come back quickly.	When they want the opportunity and you trust them to do the work — and after checking you are not just picking people like yourself.
Sharing the spotlight	A team that grows, and space for people to pick up work and lead it.	Work done less well than you would have done it, and less visible credit for you.	Constantly — but not to the point where nobody can say what you do. Managers still expect visible accomplishments.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“I'll scope it properly first, then hand it over.”	“If I wrap it neatly, every decision that would have taught them anything is already made. Hand over the mess and hold the net.”
“She's the obvious person for this.”	“She'd do an A+ job and learn nothing. Who would find it a stretch but manageable — with support I actually promise?”
“They're doing it differently from how I would.”	“Will they hit the goals? Then it's their call. I'm a guardrail here, not a second pair of hands.”

“Someone asked me about the project so I told them.”	“Then I'm still the point of contact and they're my assistant. Next time: he owns it, here's his address, he'll know better than me.”
“I gave him some good career advice.”	“Advice is cheap. What am I amplifying, boosting, connecting or defending?”
“I always end up recommending the same three people.”	“That's in-group favouritism, and it's surprisingly easy to do. Who am I not seeing?”
“I answered because I knew the answer.”	“Leave the gap. If I answer first in every meeting, nobody else gets to be the person who knows.”
“The people being promoted aren't as good as we were.”	“They need to be as good as I was when I got here, not as good as I am now. That gap is my growth.”

PART 4 · QUESTION 1

A staff engineer decides to hand over the search-relevance workstream. She spends three weeks writing a full specification, agreeing the milestones with stakeholders and resolving the two contentious decisions, then gives it to the strongest engineer on the team, who delivers it competently and says afterwards that it felt like following instructions. Over the same period she keeps answering questions about the workstream in channels — “quick one, I know the answer” — and twice rewrites parts of the design because the approach taken was not the one she had in mind, though both would have met the goals. At the end of the quarter she reports that the handover worked and that she now has capacity for something new; her manager observes that every stakeholder still comes to her.

(a) Name what she handed over, what she should have handed over instead, and the three things the alternative develops. **(b)** Assess the choice of person against the stated rule, and give the two things she should have offered whoever she picked. **(c)** The two rewrites break a specific instruction. State it, name the image used for the underlying anxiety, and give the condition under which a different approach must be accepted. **(d)** Explain why the stakeholders still come to her, name the behaviour, and give the three things that redirecting would have produced.

PART 4 · QUESTION 2

A principal engineer is asked why the same four people keep leading the visible projects. Reviewing his own year, he finds he recommended those four for every opportunity; all four joined from his previous employer and share his background. He has also, in that year, answered the first question in every team meeting, taken the incident-commander role at all six major incidents, and represented the team at every architecture forum. He does give generous public praise. When a colleague suggests he step back, he replies that he tried delegating everything two years ago and his manager complained that nobody could tell what he did. Separately he remarks that the engineers being promoted to his level are “clearly not what they used to be”.

(a) Name the bias governing the recommendations, give the term used for the industry-level version, and say what he should be doing about it. **(b)** His praise is genuine but covers only one of the four parts of sponsorship. Name all four and give a concrete action he is missing for each of the other three. **(c)** Take the meetings, the incidents and the forum together: name what he is failing to do, give the three concrete moves that address it, and state the standard for accepting somebody else's work. **(d)** His objection about delegating everything is a real warning. State it, and reconcile it with (c). Then address his closing remark.

2×2 — is it a stretch, and did you actually let go

COLUMNS: DID YOU REALLY HAND IT OVER? →

	Yes — their call, their approach, and questions go to them	**No — you still steer it and still answer for it**
A STRETCH, with a safety net	**An opportunity** Messy enough that real decisions remain, hard enough that they had to build a support system, safe enough that a wobble is recoverable. *Move:* hold the net and nothing else — coach, ask questions, inhibit the urge to step in. Then check the tell: when someone asks you, do you redirect?	**A test they cannot pass** You gave them something hard and kept the authority, so they carry the risk without the ownership that would make it worth taking. *Move:* transfer the decisions, not the labour. Name them as owner in front of the people who ask, and accept any approach that meets the goals.
Already WITHIN their reach	**Work, done** Perfectly reasonable, and not development. Anyone who can do an A+ job on a project is not going to learn from it. *Move:* fine for throughput; do not count it as growth. Next time something messy appears, look past the obvious person.	**Assistance** A tidy task, still steered by you, with the questions still arriving in your inbox. They are executing your judgement, and everyone can tell. *Move:* the costliest default, because it looks like delegation on a status report. Give away the smallest real thing completely, and stop answering for it.

Read it as: the row is *difficulty* and the column is *ownership*; only together do they make an opportunity, since difficulty without ownership is exposure and ownership without difficulty is throughput. The right-hand column is common because both its cells come from good intentions.

CARRY-OUT RULE FROM THIS PART

Give away the messy version, to someone it will stretch, and then send the questions to them — a handover that keeps the decisions, or keeps the inbox, gave away the labour and kept everything that made it worth having.

SECTION 6

Concept sketch — the whole subject, end to end

Four named groups, in the order the story runs: **ADVICE → TEACHING → GUARDRAILS → OPPORTUNITY**, each shown at its three ranges. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: bold = a group heading **reversed = the one fact to remember** **Q = retrieval cue**

The four groups rise in cost and in effect: **ADVICE** — you explain how you relate to it · **TEACHING** — they internalise it · **GUARDRAILS** — it becomes safe to try · **OPPORTUNITY** — they do it. Each runs at three ranges: **individual**, **group**, **catalyst**.

THERE ARE NOT ENOUGH HOURS TO HELP EVERYONE ONE AT A TIME.
two axes: WHAT you give, and HOW FAR it reaches.
INDIVIDUAL one person grows . GROUP many at once .
CATALYST it continues AFTER YOU STEP AWAY.
Q the two warnings attached to the tiers?
-> do NOT skip the small ones. do NOT chase the catalyst
column: too many programmes overwhelm an org, and JOINING
an existing effort usually beats founding a new one.
why bother: engineering GOES BEYOND YOURSELF . the industry
keeps CHANGING . it is right -- you are SENDING HIGH
STANDARDS INTO THE FUTURE . and (stated last) OTHER
PEOPLE'S GROWTH IS YOUR GROWTH.

GROUP 1 - ADVICE . you explain how YOU relate to the topic
IT CAN BE TAKEN OR LEFT. cheap, fast, UNTAILORED BY
CONSTRUCTION -- which is every one of its limits.
Q which limits follow from that one property?
ASKED FOR? solicited vs not. do you have context to say
something helpful AND non-obvious? if unsure, ASK.
earns the unsolicited version: they cannot see out of
it . you hold KEY INFORMATION they lack.
MENTORING IS FOCUSED ON YOUR EXPERIENCE -- not only for new
people, not only one-way, and RIGHT ADVICE CAN BE AIMED
WRONG ("be louder" may undermine someone assessed
differently).
Q the same trap in written feedback -- one example?
ANSWER THE IMPLIED QUESTION. "no" cost an hour. but do not
infodump -- and mark a comment as INTEREST or as a
REQUEST TO CHANGE COURSE.
VENT OR ADVICE? reassurance . rubber duck . war story.
BE HONEST -- name what did not work. PEER REVIEW HAS TWO
AUDIENCES; if holding back protects a promotion, say
it PRIVATELY.
GROUP: write it down . get a microphone (carry YOUR message
alongside theirs). CATALYST: advice flows without you.
start small. a talk series wants >=3 people.

GROUP 2 - TEACHING . aimed at them INTERNALISING it
SET A GOAL -- a CAPABILITY: draw the system and describe it .
send their first pull request . handle 3-5 scenarios alone.
ACTIVATE IT. their hands, their laptop. leave an ARTIFACT.
Q why is "does that make sense?" a useless check?
SPECTRUM: SHADOWING (demonstrate, be a visible role model) --
PAIRING (share knowledge, check understanding in real
time) -- REVERSE SHADOWING (they learn most here; also
a GUARDRAIL).
REVIEW TO TEACH -- six rules: understand the assignment
(draft vs pre-launch) . explain WHY not just what .
give an example of better . be clear WHAT MATTERS
(security hole / house style / preference / nitpick) .
choose your battles (passes; wrong problem beats

indentation) . IF YOU MEAN YES, SAY YES.
Q what should you do instead of 100 comments?
COACHING (vs mentoring): open questions . active listening .
making space, COUNT TO FIVE. you cannot know every
detail, so a supplied answer earns a "YES, BUT...".
GROUP: a class -- high up-front cost, amortised; goal +
exercises; or a self-paced guided walkthrough.
the LEARNING PATH: day 1 two trivial changes to learn
REVIEW; day 2 run a toy service, change the library,
see it in the monitoring. by week 2 IT WAS JUST CODE.
CATALYST: teach people to TEACH -- edit rights, honest
feedback, then LEAVE THE ROTATION. or place it in the
ONBOARDING CURRICULUM.

GROUP 3 - GUARDRAILS . not a gate. permission to move fast
A RAILING IS NOT FOR LEANING ON. it means a stumble does
not doom you -- so people go FASTER, not slower.
REVIEW AS GUARDRAIL: code . design . CHANGE MANAGEMENT.
knowing a check exists is what lets them work alone.
NEVER RUBBER-STAMP -- worse than no review, because
they already spent the confidence.
SIX CATEGORIES: should this work EXIST (a technical fix for
a conversation?) . does it SOLVE it (need AND expect) .
how does it FAIL (clean, or corrupt data / take the
money?) . is it UNDERSTANDABLE (complexity in a
well-chosen place) . does it FIT (precedent others
copy? next to a big launch?) . do the RIGHT PEOPLE
KNOW (passive voice = no actor).
Q what is the reviewer's stance, and its opposite?
PROJECT GUARDRAIL: not doing the work -- telling them they
are near a disaster they cannot recover from. for
seasoned people too. offer SUPPORT and say WHEN to call.
GROUP: PROCESSES (keep them light or people SNEAK AROUND
THEM -- make the right way the easy way; all guidelines
are wrong sometimes) . WRITTEN DECISIONS (style guides,
paved roads, policies -- USE SPARINGLY -- vision) .
ROBOTS (the timesheet on the EXIT DOOR; reminders with
links, linters, search, templates, presubmits).
CATALYST: CULTURE CHANGE -- the most effective and hardest.
UNLESS IT BECOMES CULTURE YOU WILL ALWAYS BE CHASING
COMPLIANCE. solve a real problem . choose your
battles . offer support . find allies.

GROUP 4 - OPPORTUNITY . people learn by DOING
DELEGATION: hand over the MESSY, UNSCOPED PROJECT WITH A
SAFETY NET, not a cleanly wrapped gift.
pick a STRETCH: anyone who can do an A+ job will not
learn from it. promise the support. tell them why you
think they can.
you will not get a CLONE -- inhibit the urge to step in.
GIVING AWAY YOUR LEGOS.
Q the tell that a handover was not real?
SPONSORSHIP (more active than mentoring; costs SOCIAL
CAPITAL): AMPLIFY . BOOST . CONNECT . DEFEND.
small chances COMPOUND into bigger ones.
watch IN-GROUP FAVOURITISM -- the "mirrortocracy".
CONNECT PEOPLE TO INFORMATION and you EXPAND THEIR OPTIONS.
GROUP: SHARE THE SPOTLIGHT. leave a gap . bring them to your
meeting and let them SPEAK . invite them to review. good
enough beats your version -- that is how they learn. but
do not vanish into support: managers expect visible work.
CATALYST: teach people to look past the obvious candidates.
THE STAFF ENGINEERS YOU GREW WILL GROW THE ONES WHO FOLLOW.
THE BAR: as good as YOU WERE WHEN YOU ARRIVED, not as good as
you are now. if they seem weaker, YOU HAVE GROWN.

Master 2×2 — the decision card you can carry to other problems

The two questions that survive when the vocabulary is stripped away: **does the help match the gap** — information, understanding, safety or experience — and **have you used the smallest range that would work**, rather than the widest one available?

COLUMNS: ARE YOU USING THE SMALLEST RANGE THAT WOULD WORK? →

	Yes — one person, or one team, first	**No — you reached for the programme**
The mechanism MATCHES the gap	**Well aimed** You worked out whether they were missing information, understanding, safety or experience, and you delivered it at the range the situation actually needed. *Move:* this is the whole thesis, and it is a habit rather than an achievement. Before each act of help, ask what is missing and what the smallest thing that would supply it is — then let range grow only when the need and the value are very clear.	**The right idea, over-built** A curriculum where a conversation would have done, a mentorship scheme where one relationship was needed. Too many programmes and frameworks overwhelm an organisation. *Move:* look for an existing effort to join, or convince someone else to own the structure — which still counts as being a catalyst. Teaching a class in a curriculum that exists beats founding a second one.
The mechanism MISSES the gap	**Well meant, ineffective** Advice where the gap was a skill; teaching where the person needed permission to be wrong; a review where they needed the project itself. *Move:* name the gap before choosing the tool. If they cannot yet do it, telling is not the answer; if they are afraid to try, understanding is not the answer; and if they have all three, the only thing left to give is the work.	**Expensive noise** A framework aimed at a problem nobody had, which is now everybody's overhead. It will be followed for a while, then routed around, and it will still be your job to chase it. *Move:* stop and go back to one person. The individual and group work comes first — and if the widest version is genuinely needed, it will still be needed after you have done it.

Read it as: the row is *fit* and the column is *restraint*, and the reason both are here is that the second is the one seniority erodes. The same pair separates a doctor who prescribes for the condition from one who prescribes what they know best, a teacher who sets the lesson to the class in front of them from one who reuses last year's, and a maintainer who fixes the bug from one who rewrites the subsystem.

THE THUMB RULE

Name the gap, choose the cheapest mechanism that closes it, and use the narrowest range that works. All three get skipped for the same reason: telling people things is fast, doing it at scale feels important, and both are easier than finding out what was actually missing.

SECTION 7

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

Helping one person

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
You gave good advice and it went badly for them.	Right advice, aimed wrong — your position was not their position.	Check whether the move reads the same way from where they stand. The same words make one person a leader and get another into trouble.
A mentee goes quiet after you suggest a solution.	They wanted reassurance, a rubber duck, or a war story — and unsolicited advice derails all three.	Ask whether they want to vent or want advice, then supply comfort or solutions as requested.
Someone spent an hour on a problem you could have solved in a sentence.	You answered the question asked and not the one implied.	Ask what they are trying to do. Impart what you know — without infodumping every adjacent fact.
A review comment produced a far bigger change than you intended.	The author could not tell whether you were sharing information or asking them to change course.	Say which. A bare “this other thing would also work” reads as disapproval of the whole approach.
Someone can describe the system but cannot work in it.	They were told, not taught. The session had no stated capability and their hands never touched it.	Set a goal — draw it, send the first change, handle three to five scenarios — and let them do the work, ideally leaving with an artifact.
They fix exactly what your review comment said and repeat the mistake next month.	The comment carried the what and not the why.	Explain the reasoning, or link the specific article rather than the general manual. Facts do not transfer; understanding does.
Your suggestion is met with “yes, but…”.	You supplied a solution built on details you do not have.	Switch to coaching: open questions, reflect back what you heard, and leave silence — count to five before filling it.
An engineer will not start anything without checking with you first.	No guardrail they trust, so caution is doing the work safety should be doing.	Offer to be the check — review, or be a project guardrail — and say explicitly when and how to call on you.
Someone competent delivered your project and learned nothing.	You handed over a cleanly wrapped gift, and to someone who could already do it perfectly.	Next time hand over the messy, unscoped version, to someone it will stretch, with the support promised in advance.
Stakeholders still come to you about a project you handed over.	You are proxying the questions, which makes you the point of contact and undermines the new owner.	Redirect every time: name them as owner, say you trust them to decide, and let the relationship transfer.

Helping a group, and making it last

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
You have explained the same thing three times this week.	A conversation doing a document's job.	Write it down — a set of answers to common questions, a how-to, even a channel topic. Say it once, have it read many times.
You are teaching the same class every intake and it costs you days each time.	You are stuck at the group tier; the class has no life without you.	Teach other people to teach it: give them edit rights or your blessing for variants, shadow them, give honest feedback — then leave the rotation.

New joiners are frightened of the codebase for their first month.	No learning path — they were dropped straight into the real thing.	Build one: trivial changes first, deliberately reviewed; then run a toy version and watch a real change appear in the monitoring.
People are quietly renaming work so it does not qualify for the process.	The process is too heavy. People sneak around a guardrail that costs too much.	Take weight out rather than adding enforcement. Make the right way the easy way.
Reviewers keep spending their comments on formatting.	Work a machine should be doing is landing on humans.	Have a linter enforce as much of the style guide as it can, so reviewers do not have to.
Everyone keeps forgetting a step nobody disputes.	The remembering lives in people rather than in the environment.	Put it in their way — an automated reminder carrying the link, a template with the section already in it, or automation that simply does it.
Searching for how to do something surfaces the old way.	The right way is not the findable way.	Fix the search path, updating the wrong documents with headers that point at the right one.
You spend a day a week chasing people to follow a process.	You are chasing compliance, which is what happens until the guardrail becomes culture.	Give it a real business answer, choose your battles, make the right thing easy, and find allies with influence. Then be patient.
The same few people lead every visible project.	In-group favouritism in who gets recommended.	Check who you keep picking. Look past the obvious candidates, and set the culture that open roles are posted rather than offered quietly.
You answer first in every meeting and the team stays quiet.	Overshadowing: your presence is preventing the growth it looks like it is providing.	Leave a gap or hand off the floor, bring a less senior colleague and let them speak, and accept work that is good enough rather than yours.
Nobody can say what you actually do.	The opposite failure — so much delegation that there is no visible execution.	Align with your management chain. Most managers expect direct accomplishments alongside the support work.
You are considering a third framework this quarter.	Chasing the catalyst tier. Too many programmes overwhelm an organisation.	Join an existing effort, or convince somebody else to own the new one — that still counts. Do the individual and group work first.

Reverse mapping — you see a practice, what is it there for?

The other direction. Use this when auditing how a person or an organisation develops people rather than planning one act of help: for each thing you find them doing, say what it was buying and what it does *not* buy.

WHAT YOU FIND PEOPLE DOING	WHAT IT COUNTERS	WHAT IT DOES *NOT* COVER
Asking “do you want to vent or do you want advice?”	Unsolicited advice derailing reassurance, thinking aloud, or a war story that wanted commiseration.	Whether your advice is any good for them. It settles what is wanted, not whether your answer transfers.
Publishing an article instead of telling one person	Repeating yourself, and the itch to give advice on a topic nobody asked you about.	Tailoring. Written advice reaches many people and is fitted to none of them.
Stating what someone will be able to do by the end of a session	Sessions that felt good and changed nothing, and the useless check “does that make sense?”.	Their willingness to try it alone afterwards — that is a guardrail question, not a teaching one.
Handing over the keyboard	Fluent demonstration mistaken for transferred skill; the learner watching rather than activating.	Speed. Their hands are slower than yours, and that is the price of the lesson.

Annotating review comments with how much they matter	A reader who cannot calibrate you treating a nitpick as a blocker, or a security hole as an opinion.	Volume. Marking a hundred comments still leaves a hundred comments — that needs the passes rule.
Saying “this looks good to me” on a design document	Reviewers each waiting for the others, and silence being read as an unresolved objection.	Correctness of the design. Explicit assent speeds a decision; it does not improve it.
Coaching rather than answering	Solutions built on details you do not have, and the reflexive “yes, but…” they attract.	Urgency. It is slower, and there are moments that need the answer now.
Reading every line of a change you review	The rubber stamp, which is worse than no review because the author already spent the confidence.	Your own blind spots. A careful reviewer still needs to be open to not knowing everything.
A written process for launches or incidents	The same questions being asked every time, and the growing number of ways a launch can cause an outage.	Edge cases, which will always exist — and it earns nothing if it is heavy enough to be routed around.
A style guide	Re-arguing arbitrary conventions on every new project, and inconsistent code.	Enforcement. Until a linter carries it, the cost lands on reviewers.
Automated reminders and templates	Remembering that lives in people, and reminders that depend on you sending them.	Belief. Machinery still relies on somebody having set it up and kept it alive.
Delegating an unscoped project	Handovers that teach nothing because every decision worth making was already made.	Safety on its own. Without the promised support it is not an opportunity, it is a fall.
Redirecting questions to the new owner	The slow reversal of a handover, and the new owner being made an assistant without anyone deciding to.	Their competence. It transfers authority, not the ability to answer well.
Recommending someone for a small visible project	Good work that nobody outside the team knows about, and opportunity flowing only to the obvious names.	Their appetite for it. Social capital spent on someone who does not want the chance is wasted.
Leaving a gap when a question is asked	The most senior voice answering everything and quietly preventing the growth it appears to provide.	Visibility of your own work — which managers still expect, so do not disappear entirely into support.

SECTION 8

Numbers worth remembering

NUMBER	WHAT IT IS
2 axes	What you give (four mechanisms) and how far it reaches (three tiers). Every act of influence sits somewhere on both.
4 mechanisms	Advice, teaching, guardrails and opportunity — a list of options available to you, not a checklist to complete. Play to your strengths.
3 tiers	Individual (you work so one person's skills grow), group (one effort reaches several people at once), catalyst (the change continues after you step away).
2 warnings	Do not skip the smaller tiers — review and sponsorship ripple outward, and even world-changing technologists still mentor individuals. And do not chase the catalyst tier: too many programmes and frameworks overwhelm an organisation, and joining an existing effort often beats founding a new one.
3 reasons, then a 4th	Why raising others is part of the job: engineering goes beyond yourself; the industry keeps changing and adoption needs influence; and it is right, because you are sending high standards into the future. The fourth arrives at the end — other people's growth is your growth.
10 years	The horizon in the third reason: teach your midlevel colleagues to be fantastic engineers, and think of the midlevel engineers *they* will be teaching a decade from now.
2 cases	Where unsolicited advice earns its place: when the person is in a situation they cannot see out of, and when you hold key information they do not have.
3 alternatives to advice	What someone may want instead when they describe a difficult situation: reassurance that others would also find it hard, a rubber duck to explain it to, or the chance to tell a war story. Unsolicited advice derails all three.
2 audiences	A written peer review is read by the person who asked for it and by their manager and anyone calibrating or evaluating them. If an area for growth might torpedo a deserved promotion, deliver it privately instead.
1 or 2 levels	The prompt for peer feedback when you cannot think of anything: ask yourself why this person is not one level more senior — or two — and advise on the behaviours that would get them there.
6 weeks, weekly	The kind of expectation to set at the start of a mentoring relationship, alongside agreeing what you are trying to achieve, so you do not end up staring at each other wondering what you were supposed to discuss.
3 people	The minimum to share the load when starting a recurring talk series — soliciting talks, sending reminders and watching practice runs is a bigger commitment than scheduling the meeting.
3–5 scenarios	The goal to set when teaching someone a tool or interface: name the common scenarios they should be able to handle on their own by the end of the session.
3 points	The spectrum of working together: shadowing (you do it, they watch), pairing (both, together), reverse shadowing (they do it, you watch). Not a ranking — different points suit different situations.
6 review rules	Reviewing in order to teach: understand the assignment; explain why as well as what; give an example of what would be better; be clear about what matters; choose your battles; and if you mean yes, say yes.
4 levels of comment	The annotations that let an inexperienced author calibrate you: a real hazard such as an injection attack; house style, with a link to the guide; a preference you do not feel strongly about; and an explicit nitpick.
3 coaching skills	Asking open questions, active listening, and making space. Expect not to be good at them immediately.
count to 5	If you reflexively jump into silences, count to five in your head before speaking again, so the other person can process.
day 1, day 2, week 2	The worked learning path: day one, two trivial changes deliberately given some back-and-forth so the contributor learns the review process; day two, build and run a toy client and server, watch it in production, then change the library and see the change appear in the monitoring; by week two, real changes — and the code was just code.

3 months	How long the majority of that project's contributors stayed, which is why the documented path repaid its investment quickly.
6 categories	What a guardrail review looks for: should this work exist; does it solve the problem; how will it handle failure; is it understandable; does it fit the bigger picture; do the right people know about it.
3 days	The example of being explicit about when to ask a project guardrail for help: email me if that person has not replied to you in three days, and I can be your muscle there.
3 ways to fail at writing it down	Write nothing and people complain they cannot do anything; write guidelines and they are read as law and argued at the edges; write every edge case and you get a binder of legalese that still misses situations. Hence: all guidelines are wrong sometimes.
4 written decisions	Style guides, paved roads, policies and technical vision or strategy. Use policies sparingly — edge cases always exist, and too many are simply not remembered.
5% of servers	The example of a machine-enforced guardrail: no more than this share may be rebooted at once — alongside not decommissioning a server whose on-call was recently paged.
4 aids to culture change	Solve a real problem, choose your battles, offer support, and find allies — ideally including high-level sponsors and influencers.
4 parts of sponsorship	Amplifying, boosting, connecting and defending.
3 spotlight moves	Leave a gap or hand off the floor when a question is asked; add a less senior colleague to a meeting and let them speak about their work; invite them to review designs or code connected to their work and be clear their opinion matters.
A+	The disqualifier when choosing who to delegate to: anyone who can do an A+ perfect job on a project is not going to learn from it.

Glossary

TERM	MEANING
Individual, group, catalyst	The three ranges of influence: growing one person's skills, reaching several at once, and setting up something that continues after you step away.
Advice	Explaining how you relate to a topic, to be taken or left. Cheap, scalable through writing and speaking, and untailored by construction.
Solicited / unsolicited	Whether they asked. Unsolicited advice earns its place when they cannot see out of the situation or when you hold key information they lack — and even then, ask permission.
Mentoring	Sharing your experience so someone can make use of it themselves. Focused on your experience, which is both its value and its limit.
Rubber duck debugging	Explaining a problem to someone in order to unpack it for yourself. One of the things a person may want instead of advice.
Teaching	Aiming not at the other person receiving information but at internalising it, so the test is what they can do afterwards without you.
Unlocking a topic	What the best classes do: leaving someone with a handle on something they did not have before, and sparking curiosity.
Activating knowledge	The learner doing rather than only listening — their hands, their laptop — ideally leaving with an artifact to refer back to.
Shadowing	You execute the skill while the learner watches and takes notes. Teaching by demonstrating, and a chance to model high standards.
Pairing	Working together as active participants — programming, co-authoring, whiteboarding — which lets you share knowledge and check understanding in real time.

Reverse shadowing	The learner performs the task while you watch and give notes. Where the learning is densest, and a guardrail as well.
Coaching	Teaching people to solve problems for themselves through open questions, active listening and making space — as opposed to mentoring, which supplies your experience.
“Yes, but…”	The reflexive rejection a supplied solution attracts, because the person giving it cannot know every detail of the situation.
Learning path	A documented sequence that takes a newcomer from trivial changes to real work, with the early steps chosen to teach the process rather than the domain.
Guardrail	Not for leaning on, but there to stop a stumble becoming a fall. It exists to make autonomy and exploration affordable, so people move faster.
Change management	Writing down exactly what you are going to do before you do it and having someone agree the steps — code review for command lines.
Rubber-stamping	Approving without reading. Worse than no review, because the author has already spent the confidence the review was meant to buy.
Project guardrail	The experienced presence beside somebody stretching, who does not do the work but will say when they are near an unrecoverable disaster — and who names in advance when to call for help.
Paved road	A documented set of standard, well-supported technologies teams are recommended or required not to step off, often published with each technology's standing.
Culture change	The strongest and hardest guardrail: the state in which doing otherwise would be considered weird. Until you reach it, you are chasing compliance.
Opportunity	Giving people the experiences they need to grow, on the grounds that people learn by doing more than from teaching, coaching or advice.
Giving away your Legos	The image for handing over responsibility despite the anxiety that the new person will build it wrong or take the interesting parts — and the only way to move on to bigger things.
Proxying	Answering questions about a project you handed over, which makes you the point of contact and undermines the new owner. The clearest test of whether a delegation was real.
Sponsorship	Using your position of influence to advocate for someone: amplifying, boosting, connecting, defending. More active than mentorship, and it spends your social capital.
In-group favouritism	The bias toward evaluating people more favourably when they are like you — the reason an industry that calls itself a meritocracy has been called a mirrortocracy instead.
Sharing the spotlight	Accepting good-enough work from others, leaving gaps, and making space, rather than solving everything yourself and overshadowing the team.

Answer key

PART 1 · Q1 — THE MENTEE, THE BATCH API AND THE REVIEW COMMENT

(a) When someone starts describing a difficult project, the easy and intuitive response is to say what you would do in the same situation — and it is kinder, if more difficult, to work out what they actually need. They may want help; but they may equally want **reassurance that other people would also find the problem or situation difficult**; they may be doing **rubber duck debugging**, explaining something to you so they can unpack it for themselves; or they may want to **tell a war story, to hear commiserations and congratulations for what they have navigated so far**. **Unsolicited advice derails all of those things.** The question that finds out: **“do you need space to vent, or are you looking for advice?”** — and then supply either comfort and validation or solutions, as requested. (b) The mechanism is that **the advice that worked for you might not work for someone else**. The example is exact: “just message the director and ask them to invite you to the meeting” is **an easy thing to say when the director is a peer, and much more difficult when they are your boss's boss's boss** — the move is the same, the position is not. A second example of the same failure: guidance such as “be louder in meetings” or “ask your boss for a raise” **may undermine someone else, because members of underrepresented groups are unconsciously assessed and treated differently**. Others of the same shape: the best practices of a decade ago may not work for a younger colleague now, and the social dynamics or the technology of your experience may not map onto the one your colleague is facing. (c) The engineer answered the question that was asked and not the one that was implied. **The senior person had the information and it did not occur to them to impart it** — an hour of somebody else's time, wasted. The principle: **understand what advice is being implicitly solicited, even if it is not directly asked for**, and if you are not sure, ask what your colleague is trying to do and whether they need help. What the principle explicitly does not license is **infodumping — offloading every snippet of information that could connect to the topic at hand**. (d) The author could not tell what the comment meant. **It is frustrating to get a bare comment without context about whether that means you hate the current approach**, so the author defended against the worst reading and rewrote everything. The precision rule is to **be clear about whether you are just sharing interesting information or asking them to change course** — one sentence of intent would have saved the rewrite.

PART 1 · Q2 — THREE INITIATIVES AND A PEER REVIEW

(a) The plan is aimed at the **catalyst** tier. Two warnings apply. First, **do not get too focused on chasing the catalytic types of influence**: **having too many programmes and frameworks can be overwhelming for your organisation, and you will often have more impact by taking part in an existing initiative than by setting up something new**. Second, **do the individual and group work first, and only go broader if the need and value are very clear** — and separately, do not skip the smaller tiers, since levelling colleagues up through review or sponsorship has a ripple effect. What the six repeated questions actually call for is the group tier and specifically **writing it down**: documentation means **you do not have to explain how to do something again and again**, and a set of frequently asked questions, a how-to, or even a descriptive channel topic **lets you say something once but have it read by many people**. (b) **The mentorship programme:** the administrative work **will be more time-consuming than you might expect, and this work is generally not considered to be part of an engineer's job**. The cheaper alternative is explicit — **find a manager who is interested in doing the same thing, since they will likely find it easier to frame the work as part of their job description**, and note that **convincing someone else to set up a mentorship programme still counts as being a catalyst**. **The talk series:** it is **a bigger commitment than just scheduling the meeting — you will have to solicit talks, send reminders, and possibly watch practice runs**; be clear about what you are getting into, plan for it, and **ideally start with at least three people to share the load**. **The written-culture initiative: going from an oral culture to writing everything down will not win you any friends**. Start small and look for a small but meaningful change instead — an easy-to-use documentation platform, a page of answers to the questions a team is most often interrupted by, or a quarterly documentation day with a director's endorsement. (c) Four sentences of praise fails both audiences. For **the person who asked**: assume they asked **because they genuinely want to know how to improve**, and take “what could this person do better?” literally rather than as an invitation to criticise — and if you cannot think of anything, **ask yourself why they are not one level more senior, or two, and give them advice on the behaviours to focus on to get there**. For **the manager and the calibrators**: they need the information to help your colleague grow and to notice patterns that need addressing, and silence gives them nothing. The escape hatch resolves the dilemma directly: **if you find yourself holding back on describing an area for growth because you do not want to accidentally torpedo a well-deserved promotion, consider delivering private feedback by email or in person instead** — so the growth advice reaches the person without entering the record where it could be taken out of context. (d) It is **implicit bias in how the same behaviour is described across different people**. The instruction is to **be aware of how you are describing the same behaviour**, with the paired example: **was it “consensus building” or “indecisiveness”?** A further example of the identical pattern: **were they “refreshingly down to earth” or “unprofessional”? Often it depends on who you are talking about.** The same problem underlies advice about communication style, where **“be aggressive” will make some people seem more like leaders but will get others in trouble**.

PART 2 · Q1 — THE LEDGER WALKTHROUGH AND THE SIXTY COMMENTS

(a) It failed on **goal** and on **activation**. **Teachers have a specific goal, often formalised in a lesson plan, and if you are teaching you should too** — this session had none, so "does that make sense?" was the only available check and it is answerable only by the person least able to judge. And **successful teaching includes hands-on learning and activating knowledge: the student should be doing as well as listening** — find opportunities to let them lead, whether that means using the tool themselves, typing commands, or **opening tabs on their laptop instead of yours**, and it is better still if they end up with an **artifact**, such as a diagram, to refer back to. A goal it should have had, taken from the overview case: by the end, **the joiner should be able to draw the system on a whiteboard and describe it to someone else** — which is precisely the thing that failed three weeks later. (For the codebase, the equivalent goal is everything they need to send their first pull request.) (b) **Choose your battles.** Review in several passes, first high-level comments and then in increasing detail, because **if the code does not do what it is supposed to, it does not matter whether the indentation is correct** — and the same holds for documents: **if your first comment is that the author is solving the wrong problem, it is not helpful to leave a hundred technical suggestions**. **Be clear about what matters.** When you are less experienced **it can be hard to calibrate the advice you are given** — some things are vitally important, some are nice to have and some are personal preference — so annotate so it is clear which is which. **If you mean "yes", say "yes".** Make it clear whether your comments are blocking or not and whether you are otherwise happy with the change; call out the good as well as the bad. There is also a directly applicable instruction for a change needing this much work: **the most constructive approach may not be to bury your colleague in comments — consider setting up time to talk or pair with them instead**, remembering that engineers earlier in their careers may find you intimidating and be reluctant to question your suggestions even when they think you are wrong. (c) They are waiting for each other. **When there are a lot of reviewers, each one may be hesitant to approve until the others have weighed in.** The rule requires them to break the deadlock explicitly: **if you have no objections, say so** — and in particular **explicitly say "this looks good to me" on design documents**, because code review tends to end by clicking a button to say you believe the change is safe to merge, and a document has no such button. (d) The alternative is **coaching**, which, unlike mentoring, **teaches people to solve problems for themselves**; it is slower, but **your colleague will learn much more by making their own connections than they will from you giving them the answers**. Its three skills are **asking open questions** — ones that cannot be answered yes or no, digging into the problem rather than starting with a solution; **active listening** — reflecting back what you have heard, both to check you understood and to let them hear how you frame it, since the feeling of being understood can be powerful and your framing may give them a new way to describe the problem; and **making space** — leaving enough silence for them to reflect, and **counting to five in your head before speaking again** if you jump in reflexively. The mechanism against supplying the answer: **you cannot know every detail of the situation**, so when you provide a solution **the coachee is likely to reflexively respond with a "yes, but…", a reason your advice cannot work**.

PART 2 · Q2 — THE APPRENTICES AND THE CLASS NOBODY ELSE MAY TEACH

(a) The lecture series omits any **specific goal for what the students will know or be able to do at the end**, and any **exercises or way for the students to practise what they have learned** — the two properties a class must have, in addition to the general point that a class carries **a high up-front cost the first time you teach it, which you amortise every time you teach it again**. (b) Following the worked example: **day one, two trivial changes** — for instance one adding an entry to some shared, low-stakes list and one adding their own name to the team page — with the reviewer deliberately finding a reason for **a little back-and-forth** on them. What that teaches is not the domain but **the review process**: they get used to submitting, receiving comments and responding, on work where nothing can go wrong. **Day two, build and run a small purpose-built client and server**, watch it running in production, and look at its interface and the logs and metrics it exports; then **make a local change to the library** — a new log message, or a small tweak to the logic — **and deploy it, to prove to themselves that the code still worked and that they could see their own change in the monitoring data**. What that teaches is that the system is observable and that their change is real and traceable. The result to aim for is the stated one: **by the time they get to making actual changes in week two, they are not scared of the code — it is just code**. (c) She is stuck at the **group** tier: the class exists, but it has no life without her, so the cost repeats every intake. The **catalyst** tier requires **teaching other people to teach it**. Two things she must give them: **access to edit the slides and exercises, or her blessing to create their own variants** — since **different teachers have different styles, and the instruction is to embrace that** — and **shadowing with honest feedback, because they want to learn**. Telling two willing engineers that her slides are canonical is precisely what prevents the class from acquiring a life of its own. (d) The placement move: **if the class is applicable to all your engineers, add it to an onboarding curriculum or internal learning and development path**, so that **all of your new hires will learn what you want them to know without you having to find a way to reach them** — and where a company has no such learning culture, advocating for an onboarding process, evangelising learning paths, or setting up a framework that makes it easy to create guided walkthroughs is itself the high-impact act. Once the new teachers **start teaching other people to teach the class, it has life without her**, and at that point the right move is to **remove herself from the teaching rotation**.

PART 3 · Q1 — THE RECONCILIATION SERVICE

(a) The reviewer **rubber-stamped** it. The instruction is unambiguous: **if you want to be a good guardrail, do not ever rubber-stamp changes — read carefully: every line of code, every section of a design, every step of a proposed change**. It is worse than not reviewing at all because of what a review does for the author: **when someone knows their work will be reviewed, it is easier for them to feel confident working independently**, since **they know there will be a check to make sure they do not cause an outage or spend**

months building an architecture that cannot work. The author had already adjusted their own caution to a check that did not happen — so the approval did not merely fail to add safety, it removed some. (b) **Should this work exist?** Caught. The question to ask is what problem your colleague intends to solve, and in particular **whether they are using a technical solution to solve a problem that should have been solved by talking to someone** — a month of two teams disagreeing in a channel is exactly that case. **Does this work actually solve the problem?** Caught: **does the design propose using a system in a way that will not work?** — a per-minute call against another team's export endpoint, which was never intended to be used that way. **How will it handle failure?** Caught, twice: the question is whether it will **fail in a clean way, or corrupt data or take a user's money without giving them the service they have paid for**, which is precisely the silent partial debit; and **how will you discover problems?**, which nobody could answer for a day. **Is it understandable?** Nothing in the document as described — no evidence either way about intuitive naming or whether the complexity sits in a well-chosen place. **Does it fit into the bigger picture?** Caught: **is this a risky change scheduled at the same time as a high-profile launch? Do the right people know about it?** Caught: the owning team did not know their endpoint would be called at that rate, and the failure-handling sentence has no named actor. (c) A rewrite: *"The reconciliation service will retry a failed comparison three times; if it still fails, the service will raise an alert to the payments on-call rota, and the settlements team will investigate any discrepancy that survives the retries."* The test applied is the sixth category: **are there names attached to any actions that need to happen, or is there a lot of passive voice where it is not clear which team is doing what — and do the people involved know what is expected of them?** (d) The stance: **be open to the idea that you do not know everything**, ask questions, and be constructive. What distinguishes it from gatekeeping is stated directly — **a good guardrail is not an arbitrary gatekeeper: you are on the same side as the person you are keeping safe, and you want them to succeed**.

PART 3 · Q2 — THE LAUNCH PROCEDURE NOBODY USES

(a) All three faults come from one design rule: **aim to make processes as lightweight as possible**, because **if you add a complicated procedure with lots of boilerplate, central approval and long waiting periods, it is not going to be a good guardrail — people will just sneak around it**. **Fourteen pages** is the boilerplate; **a mandatory board** is the central approval; **meeting once a week** supplies the long waiting period, and between them they added three weeks to every launch. The third fault is separate: making it a **policy** with performance-review consequences ignores the instruction to **use policies sparingly** — **it is hard to account for all the edge cases, and there will always be edge cases**, and **if there are too many policies people just will not remember all the things they are supposed to do**. And all of it inverts the purpose: a guardrail is supposed to make people **move faster when the going is safe**. (b) Renaming a release as a configuration update is **people sneaking around the process**, which is exactly what the principle predicts for a procedure of that weight. The correct response is therefore not more enforcement but less weight: **make the right way the easy way**. Note what the workaround costs the organisation — those launches now happen with no guardrail at all, so the risk has been relocated somewhere nobody is looking rather than reduced. (c) They are **chasing compliance**, which is the named consequence of not having achieved **culture change**. Processes, policies and robots scale further than being a guardrail for individuals, **but they all still rely on you doing something**; for a message to really stick **you need everyone to believe in it and care about it**, until the organisation reaches a state where **it would be considered weird to do something else**. It is **the most effective guardrail and also the most difficult to put in place**, and **unless you can make the guardrail part of your culture, you will always be chasing compliance**. Four things make the journey easier. **Solve a real problem** — align the change with what the organisation needs and expect a lot of "why" questions, so have answers that are not just aspirational: what does the business actually get out of this? **Choose your battles** — rather than a process for design review, instil a respect for design review and trust teams to choose the form; offer easy defaults and do not get hung up on everyone following exactly the same one. **Offer support** — the processes and automations should make it easy to do the right thing. **Find allies** — do not try to change the culture alone, and ideally include high-level sponsors and influencers in the organisation you are trying to change. And be patient: it takes a lot of time and dedicated effort, and it is **the only way you will ever be able to stop pushing the process along manually**. (d) Any three of: **a linter**, to **enforce as much of your style guide as it can, so reviewers do not have to** — this is the one that removes work from reviewers; **search**, arranged so that **any search for how to do something brings up the right way to do it, even if that means updating all of the "wrong" documents to have headers that point to the right place**; **templates**, so that if every design document is supposed to have a particular section, the template already includes it; **automated reminders** that put the task in someone's calendar or message them, **including a link to the process**; and **configuration checkers and presubmits** that automatically run tests or safety checks before a change lands. Better than any reminder, though, is the general form: **have an automated system do the right thing so humans do not have to**.

PART 4 · Q1 — THE SEARCH-RELEVANCE HANDOVER

(a) She handed over **a beautifully packaged, cleanly wrapped gift**, and colleagues **will not learn as much** from those. What she should have handed over is **a messy, unscoped project with a bit of a safety net**. Three things that develops: they get a chance to **hone their problem-solving abilities**, to **build their own support system**, and to **stretch their skill set** — and the reason to accept the mess deliberately is that **a messy project is a learning opportunity that is hard to get otherwise**. The engineer's own summary, that it felt like following instructions, is the predicted outcome rather than an accident. (b) Choosing the strongest engineer breaks the stated rule: **anyone who can do an A+ perfect job on a project is not going to learn from it**, and when looking for someone to delegate to you should **think**

beyond the most obvious people and **find someone who will find the work a bit of a stretch but manageable with support**. The two things to offer whoever is chosen: **promise them that support** and **describe the guardrails you can provide**; and **explain why you think they can handle the project** — bearing in mind that **they might need a little push to see the work is within their reach, since they may not yet see themselves as a project lead, and the fact that you see them that way can be a tremendous boost to their confidence**. (c) The rewrites break the instruction that **you are not going to get a clone**: **inevitably the person you have delegated to is going to take a different approach than you would have**, and the response is to **be a guardrail, coach them, and ask questions — but inhibit the urge to step in**. The image for the underlying anxiety is **giving away your Legos**: the natural worry that the new person will not build your tower in the right way, that they will take all the fun or important pieces, or that if they take over the part you were building there will be none left for you — and yet **giving away responsibility is the only way to move on to building bigger and better things**. The condition under which a different approach must be accepted is explicit: **so long as they are going to achieve the goals, let them do it their way**. (d) The stakeholders still come to her because she kept answering them. The behaviour is **proxying**: you may think you have the current state, but **answering the question makes you the point of contact, potentially undermining the colleague to whom you handed off the project** — and **you might also have the wrong answer**. Instead, **give visibility to the person who took over: note that they are the expert and owner for the topic, and show that you trust them to make decisions**. Three things that redirecting produces: **the project owner gains a connection to someone new, and any extra opportunities that come out of that connection**; **the next time that person has a question about the project, they will know where to go**; and **the project will be off your plate**.

PART 4 · Q2 — THE SAME FOUR NAMES

(a) The bias is **in-group favouritism**, described as a cognitive bias that **can let you evaluate other people more favourably if they are like you**. The industry-level term quoted against it is the **"mirrortocracy"** — the observation that an industry which talks about being a meritocracy is closer to one in which **people tend to hire people who look like themselves at greater rates than other sectors**. What he should be doing: **pay attention to who you are recommending or helping, and make sure you are not accidentally only sponsoring people who look like you — it is surprisingly easy to do**. The positive criterion remains the same: sponsor people who **want opportunities to grow and who you trust to do good work**, and who have **untapped potential worth investing in**, since you are spending social capital and should not waste it on people who do not want the opportunity or will not put in the effort. (b) The four parts are **amplifying** — promoting your colleague's good work and making sure other people know about their accomplishments, which is the part he is doing; **boosting** — recommending them for opportunities and endorsing their skills, so concretely, putting forward a name other than the usual four for the next visible project and vouching for them to whoever is choosing; **connecting** — bringing them into a network and giving them access to people they would not otherwise be able to meet, so concretely, taking someone to the architecture forum and introducing them rather than representing them; and **defending** — standing up for them when they are unfairly criticised and changing any negative perceptions of them, so concretely, correcting a mischaracterisation in a calibration discussion rather than letting it stand. Two related actions belong here: **if someone in another team is being a superstar, make sure their boss knows**, and where written peer reviews exist, **a review is a great way to make sure the good work goes on the record**. (c) He is failing to **share the spotlight**. It may feel amazing to be the resourceful and knowledgeable senior person leading the team to greatness every day, but **what you are actually doing is overshadowing the rest of the team and preventing them from growing**. The three concrete moves: **if someone asks a question in a group meeting, leave a gap or explicitly hand off the floor to another person on your team**; **add a less senior colleague to a meeting you go to, and let them speak about their work**; and **invite a less senior colleague to review designs or code that connects to their work, and be clear that their opinion matters**. The standard for accepting somebody else's work: **let other people do work that is not as good as you would have done it, so long as it is good enough — that is how they learn**. Sharing the stage also **includes delegation but means making space**: letting other people notice that work needs doing, so they can build leadership skills by picking it up or by learning to delegate it themselves. (d) The warning is real and stated: while being the face of every project limits your team, **the opposite extreme can have problems too — if you prefer to delegate everything, make sure you are aligned with your management chain about how you work, because most managers will expect some direct execution and visible accomplishments**, and you should **not put yourself into so much of a support role that nobody is quite sure what you do**. It reconciles with (c) because neither extreme is the instruction: the fix is not to delegate everything but to stop being the automatic default for every meeting, every incident and every forum, while keeping visible direct work of his own — and to agree that balance explicitly with his management chain rather than swinging between the two failures. On his closing remark: **for someone to be promoted to your level, they do not need to be as good as you are now — they need to be as good as you were when you were first promoted to the level**. So the correct reading is the other one: **if you keep seeing people join your level who are not as capable as you are, do not snark about lowered standards — think about whether that means you have grown**.

SECTION 10

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 · The four mechanisms, in ascending order of cost						
2 · The three tiers, and the two warnings attached						
3 · Three reasons this is part of the job, and the fourth						
4 · Advice's defining property, and the limits that follow						
5 · The two cases that earn unsolicited advice						
6 · Mentoring is focused on your experience — and its limit						
7 · Vent or advice — the three alternatives						
8 · Answering the implied question, without infodumping						
9 · The two audiences of a peer review, and the escape hatch						
10 · Scaling advice: writing, and a microphone						
11 · What separates teaching from advice						
12 · Setting a goal: the three worked examples						
13 · Activating knowledge, and the artifact						
14 · Shadowing, pairing, reverse shadowing — and what each buys						
15 · The six rules for reviewing in order to teach						
16 · The four levels a review comment can carry						
17 · Coaching's three skills, and the “yes, but…” mechanism						
18 · A class: up-front cost, goal, exercises						
19 · The learning path — day one, day two, week two						
20 · Teaching people to teach, and leaving the rotation						
21 · What a guardrail is for — and why it makes people faster						
22 · Why a rubber stamp is worse than no review						
23 · The six categories a guardrail review looks for						
24 · The reviewer's stance versus gatekeeping						
25 · Project guardrails, and saying when to call						
26 · Lightweight processes, and why people sneak around them						
27 · The three ways writing it down fails						
28 · The four written decisions, and the one to use sparingly						
29 · Robots and reminders — putting it in their faces						
30 · Culture change, and chasing compliance						
31 · The four things that ease culture change						

32 · Why opportunity is the strongest mechanism						
33 · The wrapped gift versus the messy project						
34 · Choosing who to delegate to, and the A+ disqualifier						
35 · You will not get a clone — and giving away your Legos						
36 · Proxying, and the three things redirecting buys						
37 · The four parts of sponsorship						
38 · In-group favouritism, and who you keep picking						
39 · Sharing the spotlight — and the opposite extreme						
40 · The bar for promotion to your level						

Final quiz — no notes, twenty minutes, write the answers

1. Name the four things you can give a colleague and the three ranges at which each can be given, and state the two cautions attached to those ranges.

2. State what distinguishes advice from every other form of help, give two limitations that follow directly from it, and name the two situations in which offering it uninvited is justified.

3. Give the three things a person may be after when they describe a hard situation to you, the sentence that establishes which, and the two readerships of written performance feedback together with the remedy when they conflict.

4. Explain what teaching aims at that advice does not, give three examples of a session objective phrased as an ability, and say why the learner must operate the keyboard and what they should take away.

5. Describe the three positions on the working-together continuum, say what each is good for, and identify which one doubles as a safety mechanism.

6. List the six principles for reviewing so that the author learns, and give the four degrees of seriousness a comment can be marked with.

7. Name the three abilities coaching depends on, explain the specific reaction a handed-over answer provokes and why, and distinguish coaching from mentoring.

8. Say what safety measures are actually for and why they increase speed, list the six things a protective review examines, and explain why approving without reading leaves someone worse off than no review would.

9. Give the rule for designing a procedure and the evasion it prevents, name the four kinds of decision that can be recorded once with the caution attached to one, and explain why altering shared belief is the strongest safety measure available.

10. State what condition a piece of work should be in when you pass it on and who should receive it, say what rules out the obvious recipient, list the four components of advocating for someone, and name the habit that quietly undoes a handover.